Computational Economics (2025) 65:3971–4013
https://doi.org/10.1007/s10614-024-10694-2

Dynamics in Realized Volatility Forecasting: Evaluating GARCH Models and Deep Learning Algorithms Across Parameter Variations

Omer Burak Akgun[1] · **Emrah Gulay**[2]

Accepted: 27 July 2024 / Published online: 12 August 2024
© The Author(s) 2024

Abstract
The modeling and forecasting of return volatility for the top three cryptocurrencies, which are identified by the highest trading volumes, is the main focus of the study. Eleven different GARCH-type models were analyzed using a comprehensive methodology in six different distributions, and deep learning algorithms were used to rigorously assess each model's forecasting performance. Additionally, the study investigates the impact of selecting dynamic parameters for the forecasting performance of these models. This study investigates if there are any appreciable differences in forecast outcomes between the two different realized variance calculations and variations in training size. Further investigation focuses on how the use of expanding and rolling windows affects the optimal window type for forecasting. Finally, the importance of choosing different error measurements is emphasized in the framework of comparing forecasting performances. Our results indicate that in GARCH-type models, 5-minute realized variance shows the best forecasting performance, while in deep learning models, median realized variance (MedRV) has the best performance. Moreover, it has been determined that an increase in the training/test ratio and the selection of the rolling window approach both play important roles in achieving better forecast accuracy. Finally, our results show that deep learning models outperform GARCH-type models in volatility forecasts.

Keywords Cryptocurrencies · Realized volatility · Volatility forecasting · Deep learning · GARCH models

✉ Emrah Gulay
emrah.gulay@deu.edu.tr

Omer Burak Akgun
Omerburak.Akgun@fibabanka.com.tr

[1] Fibabanka R & D Center, Istanbul, Türkiye

[2] Department of Econometrics, Dokuz Eylul University, 35390 Izmir, Türkiye

1 Introduction

Following the 2008 launch of Bitcoin by Satoshi Nakamoto, additional cryptocurrencies, including Ethereum, have emerged. Due to their spectacular market capitalization increase, referred to as (ElBahrawy et al., 2017), these digital assets now have a significant impact on the global financial landscape, attracting the attention of investors, regulators, public institutions, and the academic community. Cryptocurrencies exhibit heteroscedasticity, just like traditional financial time series, making it difficult to forecast their volatility, a crucial component of risk management.

The existing models of estimating and forecasting realized volatility can be mainly divided into two categories: statistical models and machine-learning models. It is known that the selection of parameters such as training set size, the type of window, and window size affect the forecasting performance in both models (Di-Giorgi et al., 2023). Additionally, it has been determined that using different RV measures affects the accuracy of volatility forecasting (Chen & Jin, 2023). The reason for selecting MedRV as an alternative to RV, as also noted by (Andersen et al., 2012), is its finite-sample robustness for jumps and small returns.

Furthermore, concerning the robustness of the models, it has been observed that the choice of training set size affects the forecasting performance in both the GARCH-type and machine learning models (Souto & Moradi, 2024). Another important issue in forecasting volatility is the preference for popular rolling and expanding windows. In this study, not only has the effective rolling window approach for addressing structural changes in the dataset been considered but also the expanding window approach, which effectively employs the information from all observations using the entire available data. Although both approaches have their unique advantages, it has been observed that different studies such as (Zhang et al., 2020; He et al., 2021) and (Gillitzer & McCarthy, 2019) choose different approaches. Besides using different forecasting window types, the literature also shows that the selection of forecasting window size leads to different forecasting performances (Inoue et al., 2017; Li et al., 2023).

In studies (Khaldi et al., 2019; Amirshahi & Lahmiri, 2023), comparing the forecasting performance of GARCH-type models with machine learning models in volatility forecasting, a limited number of GARCH-type models were considered and their performance was examined under specific error distributions. The difference between this and other studies on volatility forecasting is that it considers not only the performance of 11 different GARCH-type models using six different error distributions, but also compares the predictive power of deep learning models. This allows for investigation of more models, and the examination of the effect of many different parameters on the forecasting accuracy of the models during the modeling and forecasting processes. However, this study is limited on three fronts. The first concerns how investors and academics decide on the selection of different parameters in the process of modeling volatility. From the forecasting perspective, the models with different parameter configurations

may provide different forecasting outcomes. The second limitation concerns the financial markets, especially those dealing with cryptocurrencies like Bitcoin, Ethereum, and Binance Coin, which are very volatile and affected by multiple external factors. Unexpected events or sudden market changes may impact the accuracy of volatility forecasts which might not have been captured. To overcome this limitation and achieve more accurate forecasts, the rolling window approach has been used to capture the structural changes. The final limitation concerns the selection of data with different characteristics. The key parameters can change the forecasting results when using other financial instruments. To address this shortcoming, the returns on three different cryptocurrencies have been used.

This study contributes to the existing academic literature in three ways. First, it finds that GARCH-type models with heavy-tailed distributions can adjust well to volatility changes in three cryptocurrency prices and provide better forecast accuracy in all error metrics than the assumption of normality for errors. Second, the sensitivity of GARCH-type models and deep learning models to various parameters, such as test and training sizes, window type, window size, measures of RV, and error measure selection, provides valuable insights. This can make it easier for practitioners and academicians to comprehend how robust the predictive performance of the models is in different scenarios. Finally, using two different methods to calculate RV to test the forecasting abilities of 11 GARCH-type models with different error distributions and deep learning models gives different results, depending on the error metrics used. This range of evaluation metrics provides a way for the forecasters to understand how accurately the models forecast volatility patterns.

The remainder of the paper is organized as follows. Section 2 reviews the literature. Section 3 describes the data and methodology. Section 4 presents the empirical results and discusses the main results. Section 5 concludes the paper and outlines the future work.

2 Literature

Empirically, the volatility of cryptocurrency returns has emerged as an ongoing and significant topic in financial literature. The volatility dynamics of cryptocurrencies, especially Bitcoin, have been extensively examined, as is evident in the literature. The majority of studies in the literature currently focus on analyzing Bitcoin, while limited research targets other cryptocurrencies. It is essential to note that other digital currencies play a significant role as speculative assets.

GARCH-type models, such as GARCH (Gronwald, 2014), Threshold GARCH (TGARCH) (Bouoiyour & Selmi, 2015, 2016), and Exponential GARCH (EGARCH) (Dyhrberg, 2016), were employed to model and forecast Bitcoin's volatility. In a similar vein, Bouoiyour and Selmi (2016) compared various GARCH-type models and recommended using asymmetrical TGARCH models. In the study by Maciel (2021), the one-, five-, and ten-steps ahead forecasts for the volatility of

six leading digital coins were conducted. It was concluded that the MS-GARCH model exhibited good forecasting accuracy.

Panagiotidis et al. (2022) stated that the MS-GARCH model exhibited superior prediction accuracy compared to standard GARCH models in forecasting cryptocurrency volatility. Moreover, the significance of the EGARCH model was emphasized through the internal evaluation of the GARCH models. After researching forecasting Bitcoin price volatility, Bergsli et al. (2022) concluded that the EGARCH and APARCH models performed better. The study by Kumar et al. (2024) demonstrated that among GARCH models, the IGARCH model had the best forecasting performance for Bitcoin volatility. Fung et al. (2022) found that the asymmetric GARCH models with heavy-tailed distributions have the best goodness-of-fit properties among 254 cryptocurrencies. Naimy et al. (2021) found that different GARCH models showed varying levels of performance in forecasting the volatility of six cryptocurrencies depending on the specific cryptocurrency used.

Conversely, machine learning methodologies have been extensively used across various domains, including the forecasting of cryptocurrency prices. Although GARCH models are still the most often used model for volatility forecasting, deep learning algorithms are becoming more popular as effective and reliable alternatives. When considering machine learning models, the Artificial Neural Networks (ANN) model has been employed for forecasting financial markets by using the lags as inputs (Adhikari & Agrawal, 2014; Lahmiri, 2016, 2018; Lei, 2018). Phaladisailoed et al. (2018) have forecasted Bitcoin prices using machine learning models and deep learning methods. Recently, Akyildirim et al. (2021) has employed various machine learning methods, including support vector machines (SVM), logistic regression, ANN, and random forest, to forecast cryptocurrency returns. Mallqui and Fernandes (2019) employed the ANN and SVM models, with the findings indicating that the SVM model outperformed in out-of-sample forecasting. García-Medina and Aguayo-Moreno (2024) have shown that among GARCH models and deep learning models, multilayer perceptron (MLP) models provide the best predictive results. The study by Murray et al. (2023) investigated the forecasting performance of deep learning models for forecasting cryptocurrency prices. Their results showed that the LSTM model had the best forecasting performance for all the cryptocurrencies used.

The literature indicates that different methodologies exhibit varying degrees of superiority when analyzed in the context of modeling and forecasting price volatility. The main cause of this variation is attributed to the dynamic parameter selection made during the studies' analyses.

3 Data and Methodology

3.1 Data

In this study, data from three cryptocurrencies, namely Bitcoin, Ethereum, and Binance Coin, were utilized. As of the data acquisition date, which was in December 2021, these cryptocurrencies were among the top three in terms of market capitalization. The closing prices of these cryptocurrencies were obtained

through the Binance application programming interface (API) using the Python programming language. Two datasets were downloaded for each cryptocurrency, one containing daily data and the other containing five-minute data. While the five-minute data was solely used to calculate realized volatility, the daily data was used to construct a return series, and the models were applied using this series. The formula used to calculate the return series is provided below, where p_t represents the value of the daily closing price at time t.

$$r_t = \log\left(\frac{p_t}{p_{t-1}}\right) \tag{1}$$

The data used in the study were divided into two separate sets: training and testing. The models were trained on the training set, and their prediction performance was measured on the testing set. To investigate whether the prediction performance varied with different dataset sizes, three different train-test ratios were employed: 70:30, 80:20, and 90:10. The number of observations in both the training and testing sets for each dataset, along with other details, is further explored in this section.

For the case of Bitcoin, the daily closing prices from 18th August 2017 to 31st December 2021 were used. Figure 1 is a graph of this series. Upon examination of the graph, it can be seen that Bitcoin's price remained relatively stable until the end of 2020, after which it exhibited an increasing trend. It can also be noted that its volatility was relatively high.

In the context of Bitcoin, the 70:30 train-test ratio results in 480 observations in the testing set, the 80:20 ratio results in 320 observations, and the 90:10 ratio results in 160 observations.

Table 1, presents the descriptive statistics of the datasets used in the study. For Bitcoin, given that the mean values for both daily and five-minute data are greater than their respective median values, and the positive skewness coefficients, it can

Table 1 Descriptive statistics of three cryptocurrency datasets

	BTC			ETH			BNB		
	Daily	5-Min	Return	Daily	5-min	Return	Daily	5-min	Return
No. of Obs.	1597	458249	1596	1598	458453	1597	1517	435211	1516
Mean	17437	17424.39	0.0015	889.17	887.77	0.0015	103.8	103.71	0.0038
Std. Dev.	17285.89	17283.29	0.0429	1154.85	1153.56	0.0543	175.53	175.43	0.0643
Min.	3189.02	2820	−0.5026	83.76	82.12	−0.5905	1.49	1.35	−0.5823
Max.	67525.83	68734.26	0.203	4807.98	4862.01	0.2337	676.15	689.12	0.5324
Q1	6737.36	6718.65	−0.0159	195.92	195.41	−0.0214	12.7	12.57	−0.0236
Q2	9289.52	9281.23	0.0018	334.68	330.5	0.0015	18.61	18.63	0.002
Q3	18650.52	18545.34	0.0202	917.07	908.93	0.0299	35.02	34.91	0.03
Skewness	1.4223	1.4236	−1.0994	1.7895	1.7901	−1.1106	1.7848	1.7853	0.2817
Kurtosis	0.5167	0.5195	14.0748	2.0345	2.0339	11.362	1.7228	1.7256	14.3287

Note: Q1 is the first quartile, Q2 is the second quartile, and Q3 is the third quartile

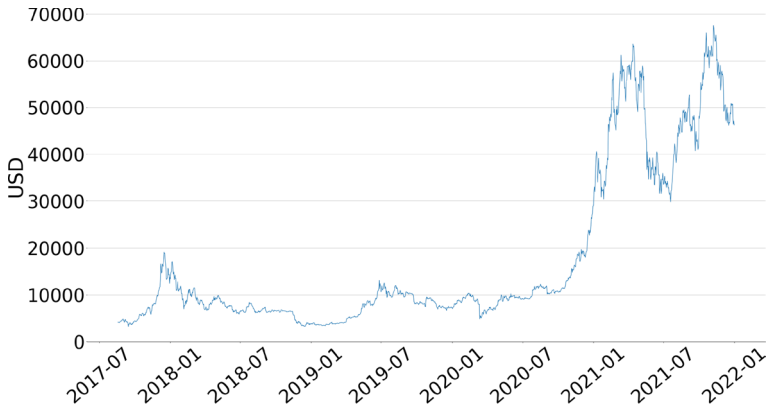


Fig. 1 The plot of the Bitcoin price series

be inferred that these distributions are right-skewed. The positive skewness values indicate that these distributions are more peaked compared to a normal distribution. Furthermore, the returns exhibit a positive mean. The negative skewness coefficient and the high kurtosis coefficient indicate that the distribution properties commonly observed in financial time series are also applicable to Bitcoin.

Line and histogram graphs of the returns are provided in Figs. 2 and 3. The line graph illustrates that times of high volatility are followed by high volatility and, similarly, times of low volatility are followed by low changes. This clustering of volatility is a characteristic feature often observed in financial time series and is a positive sign that the series can be modeled using a GARCH model.

The data for Ethereum covers the period from 17th August 2017 to 31st December 2021. The line graph of Ethereum's price, as shown in Fig. 4, indicates that it reached a peak in early 2018, followed by a decline until early 2019. Similar to Bitcoin, Ethereum exhibited a relatively stable trend until the end of 2020, after which it began an upward trend.

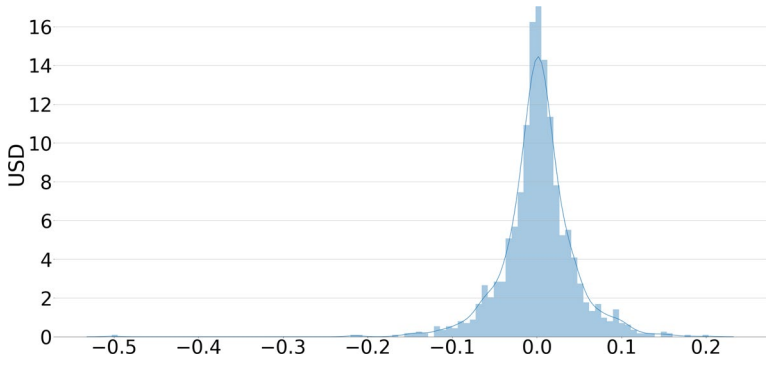


Fig. 2 Histogram of Bitcoin return series

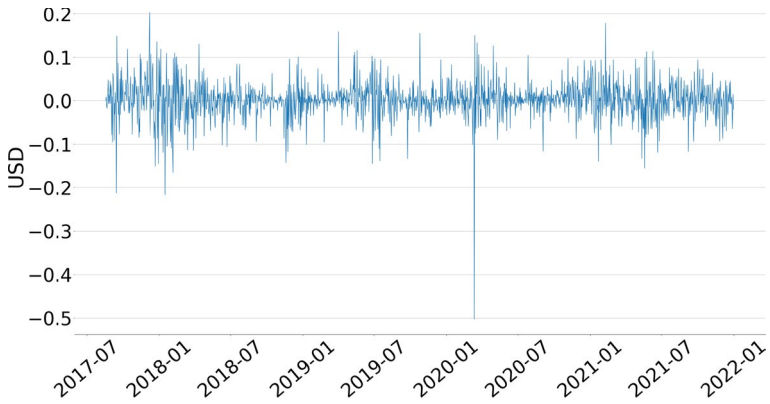


Fig. 3 The plot of Bitcoin return series

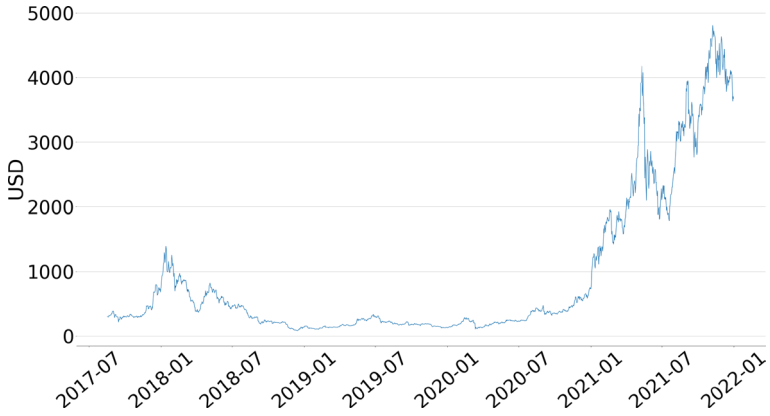


Fig. 4 The plot of the Ethereum price series

The number of observations in the testing set for Ethereum, at 70:30, 80:20, and 90:10 train-test ratios, mirrors the same values as the Bitcoin dataset: 480, 320, and 160, respectively.

Similar to Bitcoin, the mean values of the price series exceed their respective medians, and the positive skewness and kurtosis coefficients suggest that Ethereum's price distribution is right-skewed and peaked. The return series for Ethereum shares similarities with Bitcoin in terms of mean returns. However, the standard deviation of Ethereum's returns is larger than that of Bitcoin. The Ethereum return distribution also exhibits a thick-tailed characteristic.

Figures 5 and 6 present line and histogram graphs of the Ethereum return series. The line graph illustrates volatility clustering, while the histogram reflects the sharpness of the distribution and the length of the left tail, consistent with financial time series characteristics.

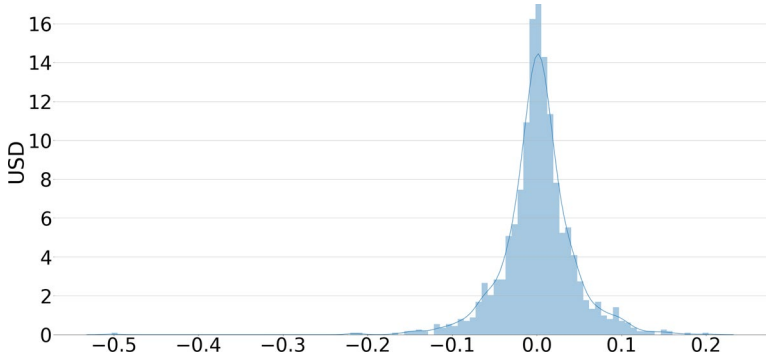


Fig. 5 Histogram of Ethereum return series

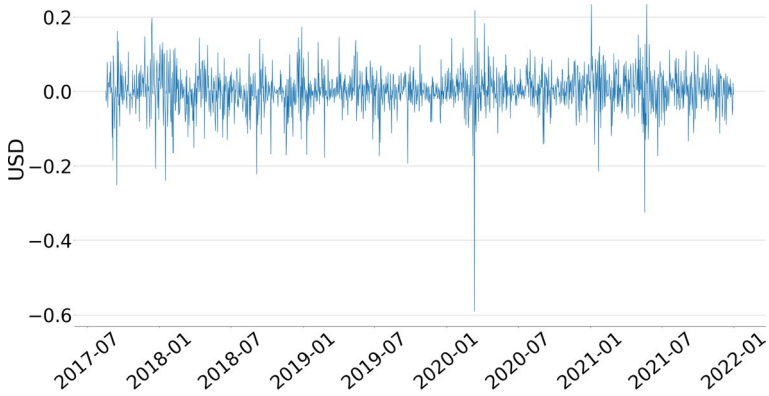


Fig. 6 The plot of Ethereum return series

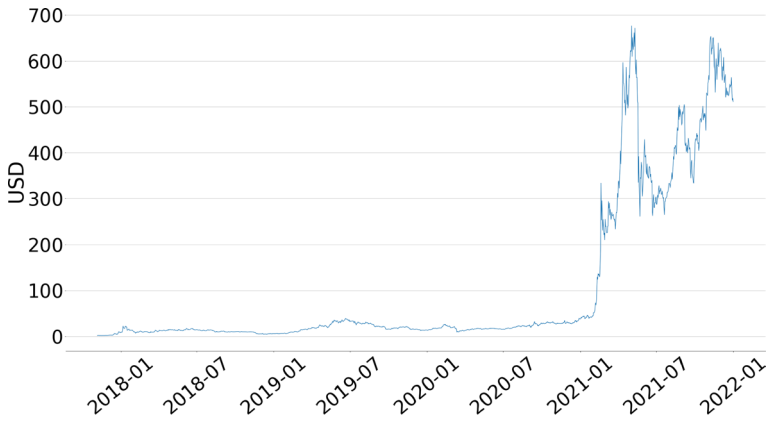


Fig. 7 The plot of the Binance Coin price series

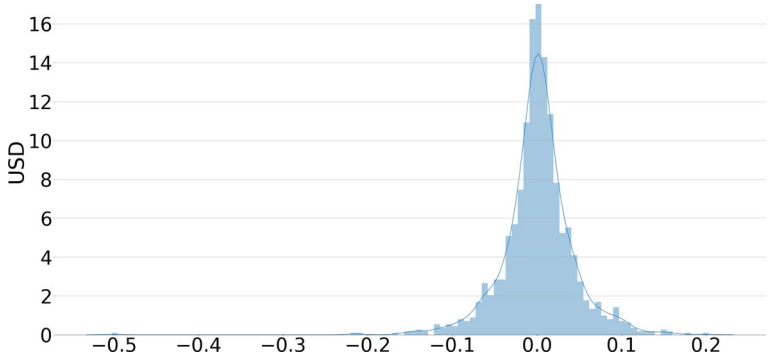


Fig. 8 Histogram of Binance Coin return series



Fig. 9 The plot of Binance Coin return series

The third and final cryptocurrency used in the study was Binance Coin, with daily closing prices covering the period from 6th November 2017 to 31st December 2021. The line graph in Fig. 7 depicts its price behavior, showing a similar pattern to the other two cryptocurrencies, with a relatively stable trend until late 2020, followed by an increasing trend.

The testing set for Binance Coin consists of 455 observations at a 70:30 ratio, 303 observations at an 80:20 ratio, and 152 observations at a 90:10 ratio. The mean values of the price series exceed their respective medians, and the positive skewness and kurtosis coefficients suggest that Binance Coin's price distribution is right-skewed and peaked. However, the return series for Binance Coin differs from the other datasets, as it exhibits larger mean and standard deviation values. Additionally, Binance Coin has the largest difference between its maximum and minimum values. The skewness coefficient suggests that this distribution is right-skewed, contrary to the others.

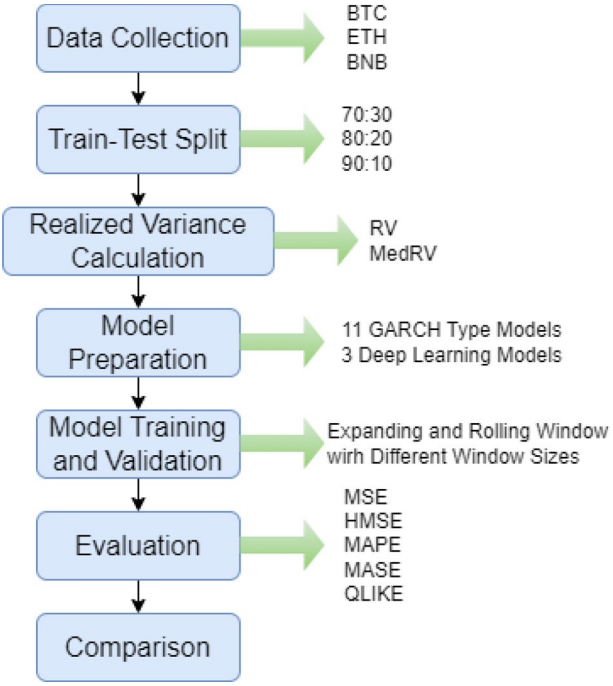


Fig. 10 Research Architecture

Figures 8 and 9 display line and histogram graphs of Binance Coin's return series, showing volatility clustering similar to Bitcoin and Ethereum. The process of empirical research discussed in Fig. 10 is summarized to better understand the impact of different parameters adapted during the modeling phase of datasets on the forecasting performance of the models.

3.2 The GARCH Models

In the study by Engle (1982), volatility is modeled as a function of past errors using the United Kingdom inflation series. With the ARCH model introduced in this work, the volatility was modeled as not constant but varying depending on past values. The return series of a financial asset can be calculated as follows:

$$r_t = \mu + \varepsilon_t \tag{2}$$

The error term ε_t can be expressed as the product of the time-dependent standard deviation term σ_t and the stochastic term z_t:

$$\varepsilon_t = \sigma_t z_t \tag{3}$$

Here, z_t is a random variable with a zero mean and unit variance. The conditional variance is given in Eq. (4):

$$\sigma_t^2 = \omega + \sum_{i=1}^{p} \alpha_i \varepsilon_{t-1}^2 \tag{4}$$

This formulation represents the equation of the ARCH model. In this equation, when $\omega > 0$ and $\alpha_i \geq 0$, the condition for the positivity of variance is satisfied. The ARCH model is effective in capturing volatility clustering.

In the study by Bollerslev (1986), the GARCH model was introduced by incorporating lagged values of conditional variance into the ARCH model. The superiority of this model over the ARCH model lies in its ability to model autocorrelation in the series with fewer parameters.

In this study, 11 different versions of the GARCH model were employed. The equations for the utilized GARCH models are provided in Table 2.

In a standard GARCH model, the assumption that the error term follows a normal distribution may not always hold true, given the characteristic features such as fat tails, high kurtosis values, and skewness, often observed in financial time series. To address this limitation, alternative distributions like the student-t distribution (std) or the generalized error distribution (ged) have been considered (Bollerslev, 1987; Nelson, 1991). This study aims to identify the most suitable distribution for the error terms in GARCH models by exploring six different distributions: normal distribution (norm), skewed normal distribution (snorm), student-t distribution, skewed student-t distribution (sstd), generalized error distribution, and skewed generalized error distribution (sged).

3.3 Deep Learning Models

3.3.1 Artificial Neural Networks

ANN is a machine learning method that was inspired by the human brain and has gained popularity over time. It was initially proposed in the study by McCulloch

Table 2 The equations of GARCH models

sGARCH	$\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2 + \beta\sigma_{t-1}^2$
IGARCH	$\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2 + (1-\alpha)\sigma_{t-1}^2$
EGARCH	$\log\left(\sigma_t^2\right) = \omega + \alpha\varepsilon_{t-1}^2 + \gamma\left(\lvert\varepsilon_{t-1}\rvert - E\left(\lvert\varepsilon_{t-1}\rvert\right)\right) + \beta\log(\sigma_{t-1}^2)$
GJR-GARCH	$\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2 + \gamma I_{t-1}\varepsilon_{t-1}^2 + \beta\sigma_{t-1}^2$
APARCH	$\sigma_t^\delta = \omega + \alpha\left(\lvert\varepsilon_{t-1}\rvert - \gamma\varepsilon_{t-1}\right)^\delta + \beta\sigma_{t-1}^\delta$
TGARCH	$\sigma_t = \omega + \alpha\left(\lvert\varepsilon_{t-1}\rvert - \gamma\varepsilon_{t-1}\right) + \beta\sigma_{t-1}$
AVGARCH	$\sigma_t = \omega + \alpha\lvert\varepsilon_{t-1}\rvert + \beta\sigma_{t-1}$
NGARCH	$\sigma_t^\delta = \omega + \alpha\lvert\varepsilon_{t-1}\rvert^\delta + \beta\sigma_{t-1}^\delta$
NAGARCH	$\sigma_t^2 = \omega + \alpha\sigma_{t-1}^2\left(\lvert\varepsilon_{t-1} - \gamma\rvert\right)^2 + \beta\sigma_{t-1}^2$
csGARCH	$\sigma_t^2 = q_t + \alpha\left(\varepsilon_{t-1}^2 - q_{t-1}\right) + \beta\left(\sigma_{t-1}^2 - q_{t-1}\right)$
	$q_t = w + pq_{t-1} + \phi\left(\varepsilon_{t-1}^2 - \sigma_{t-1}^2\right)$
ALLGARCH	$\sigma_t^\delta = \omega + \alpha\sigma_{t-1}^\delta\left(\lvert\varepsilon_{t-1} - \eta_2\rvert - \eta_1\left(\varepsilon_{t-1} - \eta_2\right)\right)^\delta + \beta\sigma_{t-1}^2$

and Pitts (1943) as a non-parametric model. A single-hidden-layer ANN can approximate any linear or non-linear continuous function (Hornik et al., 1989), making it widely used in solving classification, prediction, regression, and pattern recognition problems across various fields such as medicine, engineering, physics, biology, finance, marketing, and more.

The artificial neurons in an ANN exhibit similarities to biological neurons. In a biological neuron, the dendrites receive signals. Subsequently, these signals are sent to the cell body for processing. When the stimulation surpasses a certain threshold, the signal is transmitted to the next neuron through axons. In an artificial neuron, this threshold is modeled with the help of an activation function. Various functions, such as linear, sigmoid, hyperbolic tangent, or rectified linear units, can be selected as activation functions. The perceptron model, introduced by Rosenblatt (1958), is a single-neuron model and is the simplest form of ANN. The mathematical representation of perceptron is given in Eq. (5):

$$y = \psi\left(b + \sum_{i=1}^{n} w_i X_i\right) \tag{5}$$

In this equation, y represents the predicted value, ψ denotes the activation function, b is the bias term, w_i represents the weights learned by the model, and X_i denotes the inputs. ANN is formed by grouping artificial neurons in layers and connecting them one after the other. The outputs of the artificial neurons in the previous layer become the inputs of the artificial neurons in the next layer. In an ANN model, there are three types of layers: the input layer, hidden layer(s), and output layer. The input layer is where neurons that receive inputs from the external world are located. The hidden layer processes the data, and a model may have multiple hidden layers. The output layer is where the final prediction value is generated. An MLP is calculated as follows:

$$y = \psi\left(b_2 + \sum_{k=1}^{q} w_{ok}\theta\left(b_1 + \sum_{j=1}^{m} w_{kj}\phi\left(\ldots\ \delta\left(b_0 + \sum_{i=1}^{n} w_{li}X_i\right)\right)\right)\right) \tag{6}$$

where ψ, θ, ϕ, δ, are activation functions, b_0, b_1, and b_2 are bias terms, w_{ok} is the weight from neuron k in the last hidden layer to the output neuron. o and n are the number of inputs.

3.3.2 Long Short-Term Memory

The LSTM model, proposed in the work by Hochreiter (1997), is an extension of the recurrent neural network (RNN) model, which is used for modeling sequential data. LSTM provides a solution to the vanishing gradient problem that arises during the training of RNN models using the backpropagation algorithm (Bengio et al., 1994). This model has been successfully applied in various domains such as machine translation, speech recognition, and prediction (Goodfellow et al., 2016). LSTM incorporates gates in its structure to control the flow of information. These gates enable

the learning of which information to forget and which to remember during the training process. Sigmoid activation functions are employed in these gates, producing an output between zero and one. A zero output implies that no information passes through the gate, while a one output signifies that all information is allowed to pass. There are three types of gates in the LSTM model: input, forget, and output gates. In the input gate, the determination of which information to add to the cell state takes place. The forget gate plays a role in deciding which information should be erased from the cell state, and the output gate is responsible for determining which information should be produced as output. The LSTM model can be expressed using the following formulas:

$$f_t = \sigma\left(W_f x_t + U_f h_{t-1} + b_f\right) \tag{7}$$

$$i_t = \sigma\left(W_i x_t + U_i h_{t-1} + b_i\right) \tag{8}$$

$$\tilde{C}_t = \tanh\left(W_c x_t + U_c h_{t-1} + b_c\right) \tag{9}$$

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t \tag{10}$$

$$o_t = \sigma\left(W_o x_t + U_o h_{t-1} + b_o\right) \tag{11}$$

$$h_t = o_t \tanh\left(C_t\right) \tag{12}$$

In this given context, σ symbolizes the sigmoid function. The forget gate is denoted as f_t, the input gate as i_t, the output gate as o_t, the cell stated as C_t, the candidate cell stated as $\tilde{C}_t$, the hidden stated as h_t, and the input a time step t, as x_t. The weight matrices corresponding to the forget, input, cell state, and output gates are labeled as W_f, W_i, W_c, and W_o, while the corresponding matrices for the gates are represented by U_f, U_i, U_c, and U_o. Additionally, the bias terms associated with the forget, input, cell state, and output gates are denoted by b_f, b_i, b_c, and b_o respectively.

3.3.3 Convolutional Neural Networks

When dealing with multidimensional data such as images, ANNs tend to be computationally inefficient. Considering pixels in image data as individual variables, an ANN model requires a neuron for each variable in the input layer. Consequently, the computation involves calculating thousands of weights just in the first layer. Convolutional neural networks (CNNs) offer a more efficient computational approach to image-like data by automatically extracting features and reducing dimensions through convolution and pooling layers (Lecun et al., 1998). In the convolutional layer, a smaller-sized filter traverses the input tensor using sliding windows and conducting element-wise multiplication and summation. The result forms a new matrix, and this matrix is referred to as a feature map. Another specialized layer in the CNN architecture is the pooling

layer. Commonly situated after the convolutional layers, its primary function is subsampling. There are generally two types of pooling: max pooling and average pooling. In max pooling, a sliding window, similar to the convolutional layer, is applied, and the new matrix's element is the maximum value in the window. Similarly, in average pooling, the average of these values is calculated. The convolution layer formula is given in Eq. (13):

$$Y_{i,j} = \phi\left(\sum_{r=0}^{R-1}\sum_{c=0}^{C-1} W_{r,c}X_{i+c,j+r} + b\right) \tag{13}$$

The feature map's intersection at row i and column j is expressed as $Y_{i,j}$, and the corresponding value in the input matrix at row $i + c$ and column $j + r$ is denoted as $X_{i+c,j+r}$. The activation function applied to this operation is ϕ, and the convolution kernel's weight at row r and column c is represented as $W_{c,r}$. R and C are the height and width of the convolutional kernel, respectively. Additionally, the bias associated with the convolution kernel is denoted by b. Finally, the selection of hyperparameters used in deep learning models is provided in Table 3.

3.4 Realized Volatility

Volatility, unlike directly observable variables such as price or volume, requires the calculation of realized volatility for meaningful comparison. In the context of this study, two distinct realized volatilities, computed using five-minute data, were examined. These methodologies included the square of five-minute returns as proposed in Andersen and Bollerslev (1998) and the median realized volatility, as outlined in

Table 3 Hyperparameters of deep learning models

Parameter	ANN	LSTM	CNN
Learning rate	0.001	0.001	0.001
Batch size	30	30	30
Epochs	200	200	200
Optimizer	Adam	Adam	Adam
Activation functions in the hidden layer	Tanh	Tanh	Tanh
Loss function	MSE	MSE	MSE
Number of hidden layers	2	2	2
Number of neurons in the hidden layer	[20, 40, 60, 80,..., 200]	[20, 40, 60, 80,..., 200]	[20, 40, 60, 80,..., 200]
Lag of RV as input	[1, 2, 3,..., 10]	[1, 2, 3,..., 10]	[1, 2, 3,..., 10]
Number of filters	–	–	64
Kernel size	–	–	2
Pooling method	–	–	Max pooling
Pooling size	–	–	2

Andersen and Bollerslev (1998). The calculation involved summing the squares of five-minute returns, as per the formula provided in Eq. (14).

$$RV_t = \sum_{i=1}^{M} r_{t,i}^2 \tag{14}$$

The formula for median realized volatility is given in Eq. (15).

$$MedRV_t = \frac{\pi}{6 - 4\sqrt{3} + \pi}\left(\frac{M}{M-2}\right)\sum_{i=2}^{M-1} med\left(|r_{t,i-1}|,\ |r_{t,i}|, |r_{t,i+1}|\right)^2 \tag{15}$$

3.5 Loss Functions

The study by Patton (2011) highlighted the robustness of MSE and QLIKE loss functions in the face of noise in realized volatility calculations. In this study, the out-of-sample predictive capabilities of models were evaluated using not only these two metrics but also three additional loss functions, these being HMSE, MAPE, and MASE. The formulations for these loss functions are detailed below.

$$MSE = \frac{1}{n}\sum_{t=1}^{n}\left(\sigma_t^2 - \hat{\sigma}_t^2\right)^2 \tag{16}$$

$$HMSE = \frac{1}{n}\sum_{t=1}^{n}\left(\frac{\sigma_t^2 - \hat{\sigma}_t^2}{\sigma_t^2}\right)^2 \tag{17}$$

$$MAPE = \frac{1}{n}\sum_{t=1}^{n}\left|\frac{\sigma_t^2 - \hat{\sigma}_t^2}{\sigma_t^2}\right| \tag{18}$$

$$MASE = mean\left(\frac{|e_t|}{\frac{1}{n-1}\sum_{t=2}^{n}\left|\sigma_t^2 - \hat{\sigma}_t^2\right|}\right) \tag{19}$$

$$QLIKE = \frac{1}{n}\sum_{t=1}^{n}\left(\frac{\sigma_t^2}{\hat{\sigma}_t^2} - \log\left(\frac{\sigma_t^2}{\hat{\sigma}_t^2}\right) - 1\right) \tag{20}$$

3.6 Rolling and Expanding Window

In addition to applying the GARCH models with a fixed training set range, rolling and expanding window techniques were employed with varying window values. These techniques involved updating the model parameters at specified intervals. In

the expanding window approach, the initial point remains fixed, and the training set continuously expands. For instance, when forecasting y_t, the training set is y_0, y_1,..., y_{t-1}, and when estimating y_{t+1}, it becomes y_0, y_1,..., y_t.

Conversely, the rolling window technique maintains a constant training set size but shifts it with each prediction. For example, when forecasting y_t, the training set is y_0, y_1,..., y_{t-1}, and when forecasting y_{t+1}, it shifts to y_1, y_2,..., y_t.

4 Empirical Results and Discussion

In this study, 11 different GARCH models and three different DL algorithms were compared in terms of out-of-sample forecasting performance for each cryptocurrency, considering various parameters. During the comparisons, the impact of factors such as model type, error distribution, the rolling window method, window size, realized volatility formula and train-test ratio was also examined. Models such as sGARCH, IGARCH, EGARCH, GJR-GARCH, APARCH, TGARCH, AVGARCH, NGARCH, NAGARCH, csGARCH, and ALLGARCH were analyzed under normal, skewed normal, student-t, skewed student-t, generalized error, and skewed generalized error distributions. The ARCH (p) and GARCH (q) parameters for GARCH models were set to one. The models were applied with window sizes of 3, 5, 10, 15, 30, and 45 in both rolling and expanding window techniques, in addition to the fixed training set interval settings. They were also implemented with train-test ratios of 70:30, 80:20, and 90:10. The total number of applied GARCH models was 7722.

ANN, CNN, and LSTM were used as DL algorithms. These models use lagged values of realized volatility as input, with lag values ranging from one to 10. The DL models had two hidden layers, and the number of neurons in the hidden layers took values ranging from 20 to 200 in increments of 20. The DL models were separately applied for the squares of five-minute returns and the median realized volatility formulas. A total of 54000 DL models were considered. As a result, the total number of GARCH and DL models obtained in this study was 61722. This study utilized Python version 3.7.12 and R version 4.0.5 as the programming languages.

Tables 4, 6, 8, 10, and 12 present the frequency of selection for GARCH models and error distributions across various scenarios based on the MSE, HMSE, MAPE, MASE, and QLIKE loss functions, respectively. A comprehensive set of 234 scenarios was generated by combining different data sets, train-test ratios, rolling window methods, and realized volatility formulas. In Tables 5, 7, 9, 11, and 13, the best performing model, error distribution, rolling window and loss function values for these loss functions according to three data sets and three train-test ratios are given.

4.1 Empirical Results

Analysis of Table 4 reveals that the csGARCH model demonstrated a notable preference for the MSE loss function. Among the various error distributions

Table 4 Number of selections of GARCH models and error distributions by MSE

	norm	snorm	std	sstd	ged	sged	Total
sGARCH	0	0	0	0	1	0	1
IGARCH	10	4	0	0	1	0	15
EGARCH	2	0	12	4	16	14	48
GJR-GARCH	20	0	0	1	16	0	37
APARCH	2	0	0	0	0	3	5
TGARCH	0	0	0	0	2	2	4
AVGARCH	0	0	2	4	0	0	6
NGARCH	0	0	0	0	2	8	10
NAGARCH	0	0	0	0	0	4	4
CSGARCH	0	24	5	3	9	26	67
ALLGARCH	4	12	0	0	19	2	37
Total	38	40	19	12	66	59	234

Table 5 MSE values of selected models

Dataset	Train-test ratio	Realized vol	Model	Rolling window	MSE
BTC	70:30	MedRV	ALLGARCH(1, 1)-sged	Rolling Window 45	1.67E−05
BTC	70:30	RV	ALLGARCH(1, 1)-sged	Rolling Window 45	1.41E−05
BTC	80:20	MedRV	TGARCH(1, 1)-sged	Rolling Window 3	2.17E−05
BTC	80:20	RV	TGARCH(1, 1)-sged	Rolling Window 3	1.75E−05
BTC	90:10	MedRV	CSGARCH(1, 1)-ged	Rolling Window 45	2.39E−06
BTC	90:10	RV	ALLGARCH(1, 1)-snorm	Rolling Window 5	3.07E−06
ETH	70:30	MedRV	CSGARCH(1, 1)-ged	Rolling Window 3	1.79E−05
ETH	70:30	RV	CSGARCH(1, 1)-ged	Rolling Window 3	1.48E−05
ETH	80:20	MedRV	ALLGARCH(1, 1)-snorm	One-Step-Ahead	2.30E−05
ETH	80:20	RV	ALLGARCH(1, 1)-snorm	One-Step-Ahead	1.86E−05
ETH	90:10	MedRV	GJR-GARCH(1, 1)-sstd	Rolling Window 45	3.08E−06
ETH	90:10	RV	APARCH(1, 1)-sged	Rolling Window 45	3.57E−06
BNB	70:30	MedRV	EGARCH(1, 1)-sged	Rolling Window 30	2.76E−05
BNB	70:30	RV	EGARCH(1, 1)-sged	Rolling Window 30	2.41E−05
BNB	80:20	MedRV	EGARCH(1, 1)-sged	Rolling Window 30	3.09E−05
BNB	80:20	RV	EGARCH(1, 1)-sged	Rolling Window 30	2.56E−05
BNB	90:10	MedRV	AVGARCH(1, 1)-sstd	Rolling Window 45	3.29E−06
BNB	90:10	RV	AVGARCH(1, 1)-sstd	Rolling Window 45	3.72E−06

Table 6 Number of selections of GARCH models and error distributions by HMSE

	norm	snorm	std	sstd	ged	sged	Total
sGARCH	0	0	0	0	0	0	0
IGARCH	0	0	0	11	4	0	15
EGARCH	0	0	8	17	0	0	25
GJR-GARCH	1	0	8	0	1	0	10
APARCH	0	1	0	0	0	0	1
TGARCH	3	0	9	3	8	22	45
AVGARCH	13	22	10	13	15	8	81
NGARCH	0	1	13	1	0	0	15
NAGARCH	15	0	0	0	0	0	15
CSGARCH	2	0	8	3	1	0	14
ALLGARCH	1	3	0	8	1	0	13
Total	35	27	56	56	30	30	234

Table 7 HMSE values of selected models

Dataset	Train-test ratio	Realized vol	Model	Rolling window	HMSE
BTC	70:30	MedRV	EGARCH(1, 1)-sstd	Rolling Window 5	0.9682
BTC	70:30	RV	EGARCH(1, 1)-sstd	Rolling Window 5	0.9668
BTC	80:20	MedRV	AVGARCH(1, 1)-snorm	Rolling Window 10	0.9923
BTC	80:20	RV	NAGARCH(1, 1)-norm	Rolling Window 15	0.9911
BTC	90:10	MedRV	AVGARCH(1, 1)-snorm	Rolling Window 45	0.7199
BTC	90:10	RV	AVGARCH(1, 1)-snorm	Rolling Window 45	0.7288
ETH	70:30	MedRV	EGARCH(1, 1)-sstd	Rolling Window 3	0.9752
ETH	70:30	RV	EGARCH(1, 1)-sstd	Rolling Window 3	0.9755
ETH	80:20	MedRV	TGARCH(1, 1)-sged	Rolling Window 5	0.9912
ETH	80:20	RV	TGARCH(1, 1)-sged	Rolling Window 5	0.9912
ETH	90:10	MedRV	GJR-GARCH(1, 1)-std	Rolling Window 5	0.6078
ETH	90:10	RV	GJR-GARCH(1, 1)-std	Rolling Window 5	0.6401
BNB	70:30	MedRV	EGARCH(1, 1)-sstd	One-Step-Ahead	0.6087
BNB	70:30	RV	EGARCH(1, 1)-sstd	One-Step-Ahead	0.6091
BNB	80:20	MedRV	AVGARCH(1, 1)-sged	Rolling Window 45	0.9859
BNB	80:20	RV	TGARCH(1, 1)-ged	Rolling Window 45	0.9867
BNB	90:10	MedRV	AVGARCH(1, 1)-sged	Rolling Window 45	0.5952
BNB	90:10	RV	AVGARCH(1, 1)-sged	Rolling Window 45	0.5663

Table 8 Number of selections of GARCH models and error distributions by MAPE

	norm	snorm	std	sstd	ged	sged	Total
sGARCH	0	0	0	0	11	6	17
IGARCH	0	0	0	0	0	0	0
EGARCH	4	0	0	0	4	0	8
GJR-GARCH	2	0	0	6	14	2	24
APARCH	0	0	12	8	0	2	22
TGARCH	0	0	0	0	0	0	0
AVGARCH	0	0	0	8	5	0	13
NGARCH	0	0	0	0	8	16	24
NAGARCH	0	0	0	0	2	4	6
CSGARCH	0	2	29	2	21	33	87
ALLGARCH	6	0	0	2	21	4	33
Total	12	2	41	26	86	67	234

Table 9 MAPE values of selected models

Dataset	Train-test ratio	Realized vol	Model	Rolling window	MAPE
BTC	70:30	MedRV	CSGARCH(1, 1)-sged	Rolling Window 45	0.6411
BTC	70:30	RV	CSGARCH(1, 1)-sged	Rolling Window 45	0.539
BTC	80:20	MedRV	NGARCH(1, 1)-sged	Rolling Window 45	0.6085
BTC	80:20	RV	NGARCH(1, 1)-sged	Rolling Window 45	0.5103
BTC	90:10	MedRV	GJR-GARCH(1, 1)-sstd	Rolling Window 45	0.6275
BTC	90:10	RV	GJR-GARCH(1, 1)-sstd	Rolling Window 45	0.5232
ETH	70:30	MedRV	GJR-GARCH(1, 1)-sged	One-Step-Ahead	1.7581
ETH	70:30	RV	GJR-GARCH(1, 1)-sged	One-Step-Ahead	1.4758
ETH	80:20	MedRV	GJR-GARCH(1, 1)-sstd	Rolling Window 30	1.3852
ETH	80:20	RV	GJR-GARCH(1, 1)-sstd	Rolling Window 30	1.1612
ETH	90:10	MedRV	APARCH(1, 1)-std	Rolling Window 45	1.2989
ETH	90:10	RV	APARCH(1, 1)-std	Rolling Window 45	1.0755
BNB	70:30	MedRV	AVGARCH(1, 1)-sstd	Expanding Window 5	2.4214
BNB	70:30	RV	EGARCH(1, 1)-norm	Rolling Window 3	2.0909
BNB	80:20	MedRV	ALLGARCH(1, 1)-norm	Expanding Window 3	2.1971
BNB	80:20	RV	ALLGARCH(1, 1)-norm	Expanding Window 3	1.9127
BNB	90:10	MedRV	AVGARCH(1, 1)-sstd	Rolling Window 45	1.3565
BNB	90:10	RV	AVGARCH(1, 1)-sstd	Rolling Window 45	1.1449

Table 10 Number of selections of GARCH models and error distributions by MASE

	norm	snorm	std	sstd	ged	sged	Total
sGARCH	0	0	0	0	0	0	0
IGARCH	9	0	0	0	0	0	9
EGARCH	12	2	0	8	4	2	28
GJR-GARCH	0	0	0	0	0	2	2
APARCH	6	0	0	0	0	0	6
TGARCH	16	0	0	0	8	0	24
AVGARCH	46	4	0	0	6	2	58
NGARCH	0	0	0	0	0	0	0
NAGARCH	0	0	0	0	0	0	0
CSGARCH	2	0	4	4	8	13	31
ALLGARCH	57	19	0	0	0	0	76
Total	148	25	4	12	26	19	234

Table 11 MASE values of selected models

Dataset	Train-test ratio	Realized vol	Model	Rolling window	MASE
BTC	70:30	MedRV	EGARCH(1, 1)-sstd	Rolling Window 3	2.0272
BTC	70:30	RV	EGARCH(1, 1)-sstd	Rolling Window 3	2.0135
BTC	80:20	MedRV	ALLGARCH(1, 1)-norm	One-Step-Ahead	3.5302
BTC	80:20	RV	ALLGARCH(1, 1)-norm	One-Step-Ahead	3.4785
BTC	90:10	MedRV	ALLGARCH(1, 1)-snorm	One-Step-Ahead	4.2692
BTC	90:10	RV	ALLGARCH(1, 1)-snorm	One-Step-Ahead	4.2537
ETH	70:30	MedRV	EGARCH(1, 1)-sstd	Rolling Window 3	1.8005
ETH	70:30	RV	EGARCH(1, 1)-sstd	Rolling Window 3	1.783
ETH	80:20	MedRV	ALLGARCH(1, 1)-norm	Expanding Window 3	4.5373
ETH	80:20	RV	ALLGARCH(1, 1)-norm	Expanding Window 3	4.3048
ETH	90:10	MedRV	AVGARCH(1, 1)-ged	Rolling Window 45	4.2041
ETH	90:10	RV	AVGARCH(1, 1)-ged	Rolling Window 45	4.0816
BNB	70:30	MedRV	CSGARCH(1, 1)-ged	Expanding Window 3	0.6173
BNB	70:30	RV	CSGARCH(1, 1)-ged	Expanding Window 3	0.6173
BNB	80:20	MedRV	ALLGARCH(1, 1)-norm	Expanding Window 3	3.8155
BNB	80:20	RV	ALLGARCH(1, 1)-norm	Expanding Window 3	3.6531
BNB	90:10	MedRV	TGARCH(1, 1)-norm	Rolling Window 10	3.539
BNB	90:10	RV	TGARCH(1, 1)-norm	Rolling Window 10	3.4086

Table 12 Number of selections of GARCH models and error distributions by QLIKE

	norm	snorm	std	sstd	ged	sged	Total
sGARCH	0	0	0	0	4	8	12
IGARCH	2	0	0	0	1	0	3
EGARCH	5	0	0	0	13	11	29
GJR-GARCH	12	2	1	0	14	1	30
APARCH	0	0	8	2	2	6	18
TGARCH	0	0	0	3	0	1	4
AVGARCH	0	0	0	1	0	15	16
NGARCH	0	0	0	4	4	8	16
NAGARCH	0	0	0	0	2	4	6
CSGARCH	4	2	0	0	4	30	40
ALLGARCH	5	22	0	0	21	12	60
Total	28	26	9	10	65	96	234

Table 13 QLIKE values of selected models

Dataset	Train-test ratio	Realized vol	Model	Rolling window	QLIKE
BTC	70:30	MedRV	ALLGARCH(1, 1)-sged	Rolling Window 45	0.3021
BTC	70:30	RV	ALLGARCH(1, 1)-sged	Rolling Window 45	0.2972
BTC	80:20	MedRV	GJR-GARCH(1, 1)-ged	Rolling Window 10	0.3184
BTC	80:20	RV	ALLGARCH(1, 1)-norm	Expanding Window 10	0.3084
BTC	90:10	MedRV	ALLGARCH(1, 1)-snorm	Rolling Window 15	−0.0114
BTC	90:10	RV	ALLGARCH(1, 1)-snorm	Rolling Window 5	−0.0111
ETH	70:30	MedRV	GJR-GARCH(1, 1)-sged	One-Step-Ahead	0.4235
ETH	70:30	RV	NGARCH(1, 1)-sstd	Rolling Window 5	0.3786
ETH	80:20	MedRV	APARCH(1, 1)-std	Rolling Window 45	0.3895
ETH	80:20	RV	NAGARCH(1, 1)-ged	Rolling Window 15	0.347
ETH	90:10	MedRV	APARCH(1, 1)-sged	Rolling Window 45	−0.0112
ETH	90:10	RV	APARCH(1, 1)-sged	Rolling Window 45	−0.011
BNB	70:30	MedRV	EGARCH(1, 1)-ged	One-Step-Ahead	0.5525
BNB	70:30	RV	EGARCH(1, 1)-ged	One-Step-Ahead	0.4998
BNB	80:20	MedRV	GJR-GARCH(1, 1)-ged	Rolling Window 5	0.5323
BNB	80:20	RV	EGARCH(1, 1)-sged	Rolling Window 30	0.4841
BNB	90:10	MedRV	AVGARCH(1, 1)-sstd	Rolling Window 45	−0.0112
BNB	90:10	RV	ALLGARCH(1, 1)-sged	Rolling Window 45	−0.011

considered, the ged stood out as the most commonly chosen distribution. However, it is noteworthy that within the specific framework of the csGARCH model, the sged gained prominence as the preferred choice.

According to the results presented in Table 5, it can be seen that the ALLGARCH model, employed with sged error distribution and a rolling window of 45 in the Bitcoin dataset at a 70:30 train-test ratio, demonstrated the optimal performance with values of 0.000017 and 0.0000141 for both realized volatility formulas. Similarly, for the 80:20 ratio, the TGARCH-sged model was selected, while for the 90:10 ratio, the csGARCH-ged and ALLGARCH-snorm models were chosen. The performance of the selected models for other scenarios can be seen in Table 5.

When the results are examined based on the HMSE loss function, it can be seen that the most frequently chosen model was AVGARCH, and the most frequently selected distributions were std and sstd. The AVGARCH model yielded the best results 22 times when modeled with the snorm error distribution. This number was equal to the number of selections when the TGARCH model was modeled under the sged distribution. The sGARCH model, on the other hand, demonstrated notably poor performance, both in terms of MSE and HMSE, being the least frequently chosen model. Furthermore, Table 6 shows that another model exhibiting poor performance was the APARCH model.

Additionally, in Table 7, it is evident that the AVGARCH model was the selected model in scenarios involving BTC 80:20, BTC 90:10, BNB 80:20, and BNB 90:10.

The results obtained from the MAPE were similar to those obtained by MSE. As can be seen in Table 8, the csGARCH model demonstrated superior performance in 87 out of 234 scenarios, while the ged distribution yielded the best results in 86 instances. In terms of model distribution combinations, the csGARCH and sged combination was selected more frequently than other combinations. Contrary to the outcomes in MSE and HMSE, the sGARCH models did not emerge as the worst models according to MAPE; instead, the IGARCH and TGARCH models exhibited the poorest performance. Table 8 indicates that the snorm distribution corresponded to the least favorable error distribution.

Based on the MAPE values presented in Table 9, the csGARCH-sged model was chosen for the BTC 70:30 scenario, the NGARCH-sged model for the 80:20 scenario, and the GJR-GARCH-sstd model for the 90:10 scenario. In the Ethereum dataset, the GJR-GARCH model was the optimal choice in four out of six scenarios, while the APARCH model was favored in the remaining two scenarios. ALLGARCH, AVGARCH and EGARCH models were among the selected models in the Binance Coin data set.

When the results are examined according to MASE, a different pattern emerges compared to the previous findings. While the ALLGARCH model yields the best results in 76 scenarios, the norm distribution, characterized by the best-performing error distribution, offers the most favorable distribution in more than half of the total scenarios. In contrast to the previous results, the sGARCH model has not been selected in any scenarios. The models not chosen include the NGARCH and NAGARCH models.

Table 14 Loss functions of rolling window methods on BTC dataset with a 70:30 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	0.9915	0.583	3.6054	1.44E−05	0.3057	0.9921	0.684	3.5834	1.71E−05	0.3306
Rolling W. 3	0.9713	0.6204	**2.0135**	1.44E−05	0.3169	0.9728	0.7434	**2.0272**	1.72E−05	0.3395
Rolling W. 5	**0.9668**	0.6201	3.1570	1.43E−05	0.3133	**0.9682**	0.7431	3.1748	1.71E−05	0.3401
Rolling W. 10	0.9911	0.5612	5.0415	1.41E−05	0.3197	0.9916	0.6684	5.1246	1.69E−05	0.3382
Rolling W. 15	0.9910	0.5484	4.862	1.46E−05	0.3209	0.9913	0.6541	4.9489	1.72E−05	0.3258
Rolling W. 30	0.9909	0.5444	5.4782	1.44E−05	0.3151	0.9913	0.6494	5.554	1.71E−05	0.3193
Rolling W. 45	0.9901	**0.5390**	5.5429	**1.41E−05**	**0.2972**	0.9904	**0.6411**	5.5533	**1.67E−05**	**0.3021**
Expanding W. 3	0.9929	0.6475	4.4981	1.44E−05	0.3122	0.995	0.7764	4.591	1.72E−05	0.3323
Expanding W. 5	0.9929	0.6476	4.5039	1.44E−05	0.3225	0.995	0.7765	4.5415	1.72E−05	0.3429
Expanding W. 10	0.9929	0.6482	4.3565	1.44E−05	0.3225	0.9949	0.7768	4.3798	1.71E−05	0.3429
Expanding W. 15	0.9929	0.6482	4.4795	1.44E−05	0.3097	0.995	0.7771	4.5724	1.72E−05	0.3297
Expanding W. 30	0.9929	0.6492	4.4432	1.44E−05	0.3103	0.995	0.778	4.5351	1.72E−05	0.3311
Expanding W. 45	0.9929	0.6507	4.3870	1.44E−05	0.3230	0.995	0.7806	4.4764	1.71E−05	0.3474

Bold values indicate the minimum values of error metrics

Table 15 Loss functions of rolling window methods on BTC dataset with a 80:20 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	0.9927	0.6024	**3.4785**	1.79E−05	0.3094	0.9935	0.7256	**3.5302**	2.21E−05	0.3207
Rolling W. 3	0.9914	0.5218	3.9020	**1.75E−05**	0.3328	0.9924	0.6235	4.0645	**2.17E−05**	0.3375
Rolling W. 5	0.9913	0.5157	4.1848	1.79E−05	0.3189	0.9924	0.6141	4.3676	2.20E−05	0.3198
Rolling W. 10	0.9914	0.5196	4.9819	1.78E−05	0.3186	**0.9923**	0.6210	5.1289	2.20E−05	**0.3184**
Rolling W. 15	**0.9911**	0.5205	4.9149	1.79E−05	0.3338	0.9924	0.6209	5.0451	2.20E−05	0.3374
Rolling W. 30	0.9912	0.5165	5.0499	1.79E−05	0.3294	0.9925	0.6175	5.1992	2.21E−05	0.3325
Rolling W. 45	0.9913	**0.5103**	5.0052	1.79E−05	0.3338	0.9926	**0.6085**	5.1338	2.21E−05	0.3377
Expanding W. 3	0.9927	0.6016	4.7028	1.80E−05	0.3110	0.9941	0.7236	4.7315	2.21E−05	0.3215
Expanding W. 5	0.9927	0.6017	4.7624	1.80E−05	0.3102	0.9941	0.7238	4.7968	2.21E−05	0.3208
Expanding W. 10	0.9927	0.6019	4.5465	1.80E−05	**0.3084**	0.9941	0.7242	4.5775	2.21E−05	0.3212
Expanding W. 15	0.9927	0.6025	4.5532	1.80E−05	0.3104	0.9941	0.7250	4.5859	2.21E−05	0.3212
Expanding W. 30	0.9927	0.6027	4.4592	1.80E−05	0.3104	0.9941	0.7249	4.5030	2.21E−05	0.3211
Expanding W. 45	0.9927	0.6034	4.3629	1.80E−05	0.3103	0.9941	0.7259	4.4111	2.21E−05	0.3212

Bold values indicate the minimum values of error metrics

The results presented in Table 10 parallel those in Table 11. The ALLGARCH model has proven to be the best-performing model in scenarios such as BTC 80:20 and 90:10, ETH 80:20, and BNB 80:20.

Finally, when the results are scrutinized using the QLIKE loss function, similar to the findings in MASE, the ALLGARCH model is prominent. The subsequent best-performing model is the csGARCH model, which yields the best results for MSE and MAPE. The csGARCH model, when modeled using sged error distribution, outperforms other models according to these three loss functions. When considering all the loss functions, it can be seen that different models and error distributions come to the forefront. However, among the applied 11 GARCH models, the csGARCH and ALLGARCH models are more frequently selected compared to other models, indicating their superior performance.

The ALLGARCH model yielded the optimal results in five out of six models within the BTC dataset, whereas in the ETH dataset, the GJR-GARCH, NGARCH, APARCH, and NAGARCH models are among the selected models. In the case of BNB, involving the EGARCH, GJR-GARCH, and AVGARCH, Table 12 shows that the ALLGARCH models were among the models selected in different scenarios. These scenarios and the corresponding QLIKE values can be seen in Table 13.

Upon scrutiny of the results up to this point, it seems challenging to identify a superior model or error distribution that is consistent across all scenarios. The optimal model appears to vary based on the utilized dataset, the train-test ratio, the rolling window technique, the realized volatility formula, and the loss function applied.

Within the scope of this study, another question to be addressed is which rolling window method and window size are more suitable for forecasting the volatility of cryptocurrencies. For this, models were applied with 13 different rolling window approaches. The results for the various data sets and train-test ratios are presented in Tables 14 through 22. Additionally, the interpretation of whether the results vary according to the formula for realized volatility can be gleaned from these tables.

The results obtained utilizing a 70:30 train-test ratio in the Bitcoin dataset are presented in Table 14. According to this table, the expanding window method was not selected. The results for the squares of five-minute returns and median realized volatility formulas remain unchanged. Additionally, in three out of five loss functions, the rolling window 45 has been chosen.

In the scenario modeled with an 80:20 train-test ratio in Bitcoin, the results exhibit some variations. In Table 15, the results exhibit consistency across the two RV formulas, as indicated by MAPE, MASE, and MSE. However, this consistency is not observed concerning HMSE and QLIKE.

In the final scenario of the Bitcoin dataset, the training set encompasses 90% of the entire data and the rest is reserved for testing. Table 16 reveals that, for the first realized volatility formula based on the square of five-minute returns, rolling window 45 emerged as the optimal method in terms of HMSE and MAPE loss functions. Similarly, for the median realized volatility formula, rolling Window 45 demonstrated superiority in HMSE, MAPE, and MSE loss functions. Interestingly, the MASE loss function identified the method employing a fixed training set interval as yielding the best results in both formulas. However, it is noteworthy that the MSE

Table 16 Loss functions of rolling window methods on BTC dataset with a 90:10 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	0.9944	0.5632	**4.2537**	3.16E−06	0.2612	0.9954	0.6490	**4.2692**	2.45E−06	0.2765
Rolling W. 3	0.9947	0.6486	4.3173	3.08E−06	0.2420	0.9957	0.7820	4.3387	2.46E−06	0.2565
Rolling W. 5	1.1610	0.6488	4.3539	**3.07E−06**	−0.0111	0.9809	0.7820	4.3870	2.44E−06	−0.0114
Rolling W. 10	1.1615	0.6485	4.3748	3.10E−06	−0.0111	0.9824	0.7819	4.4104	2.47E−06	−0.0114
Rolling W. 15	1.1467	0.6478	4.2605	3.09E−06	−0.0111	0.9701	0.7817	4.2866	2.45E−06	**−0.0114**
Rolling W. 30	1.1509	0.6423	4.4420	3.14E−06	−0.0111	0.9745	0.7814	4.4647	2.47E−06	−0.0113
Rolling W. 45	**0.7288**	**0.5232**	4.3760	3.12E−06	−0.0111	**0.7199**	**0.6275**	4.4466	**2.39E−06**	−0.0114
Expanding W. 3	0.9947	0.6150	4.4384	3.16E−06	0.2553	0.9955	0.7572	4.4599	2.45E−06	0.2711
Expanding W. 5	0.9947	0.6151	4.4015	3.16E−06	0.2537	0.9955	0.7574	4.4131	2.45E−06	0.2689
Expanding W. 10	0.9947	0.6154	4.4027	3.16E−06	**0.2549**	0.9955	0.7577	4.4173	2.45E−06	0.2702
Expanding W. 15	0.9947	0.6156	4.3877	3.16E−06	0.2552	0.9956	0.7580	4.4129	2.45E−06	0.2711
Expanding W. 30	0.9947	0.6159	4.3084	3.16E−06	0.2553	0.9956	0.7584	4.3217	2.45E−06	0.2708
Expanding W. 45	0.9947	0.6179	4.2853	3.16E−06	0.2558	0.9954	0.7608	4.2993	2.45E−06	0.2717

Bold values indicate the minimum values of error metrics

Table 17 Loss functions of rolling window methods on ETH dataset with a 70:30 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	0.9918	**1.4758**	4.9114	1.49E−05	0.3824	0.9911	**1.7581**	5.1757	1.81E−05	**0.4235**
Rolling W. 3	**0.9755**	1.5954	**1.7830**	**1.48E−05**	0.3925	**0.9752**	1.9059	**1.8005**	**1.79E−05**	0.4442
Rolling W. 5	0.9904	1.5398	4.7970	1.51E−05	**0.3786**	0.9904	1.8361	5.0115	1.82E−05	0.4251
Rolling W. 10	0.9934	1.6832	4.2933	1.48E−05	0.4007	0.9931	2.0093	4.5202	1.79E−05	0.4541
Rolling W. 15	0.9906	1.5471	4.9759	1.51E−05	0.3813	0.9905	1.8457	5.2044	1.82E−05	0.4278
Rolling W. 30	0.9907	1.5341	4.9952	1.50E−05	0.3799	0.9907	1.8303	5.2025	1.81E−05	0.4267
Rolling W. 45	0.9910	1.5348	5.0331	1.50E−05	0.3793	0.9909	1.8310	5.2663	1.81E−05	0.4264
Expanding W. 3	0.9933	1.6782	4.8007	1.49E−05	0.3926	0.9930	2.0046	5.0590	1.80E−05	0.4464
Expanding W. 5	0.9933	1.6845	4.8013	1.48E−05	0.3919	0.9930	2.0131	5.0580	1.80E−05	0.4448
Expanding W. 10	0.9933	1.6949	4.8279	1.49E−05	**0.3936**	0.9930	2.0243	5.0796	1.81E−05	0.4466
Expanding W. 15	0.9933	1.6936	4.7772	1.50E−05	0.3994	0.9930	2.0225	5.0323	1.81E−05	0.4522
Expanding W. 30	0.9933	1.6988	4.7730	1.49E−05	0.4000	0.9930	2.0287	5.0228	1.80E−05	0.4544
Expanding W. 45	0.9933	1.6720	4.7497	1.48E−05	0.3951	0.9930	1.9988	5.0008	1.79E−05	0.4489

Bold values indicate the minimum values of error metrics

Table 18 Loss functions of rolling window methods on ETH dataset with a 80:20 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	0.9924	1.3076	4.8595	**1.86E−05**	0.3641	0.9920	1.5453	5.1505	**2.30E−05**	0.4131
Rolling W. 3	0.9915	1.1896	4.3925	1.92E−05	0.3488	0.9913	1.4158	4.6259	2.38E−05	0.3908
Rolling W. 5	0.9912	1.1840	4.3750	1.91E−05	0.3488	**0.9912**	1.4105	4.6106	2.37E−05	0.3924
Rolling W. 10	0.9913	1.1890	4.3659	1.91E−05	0.3505	0.9913	1.4159	4.6082	2.38E−05	0.3930
Rolling W. 15	0.9913	1.1901	4.4461	1.90E−05	**0.3470**	0.9913	1.4151	4.6879	2.36E−05	0.3916
Rolling W. 30	0.9913	**1.1612**	4.4099	1.89E−05	0.3490	0.9913	**1.3852**	4.6441	2.36E−05	0.3901
Rolling W. 45	**0.9912**	1.1765	4.3910	1.90E−05	0.3480	0.9912	1.4000	4.6290	2.36E−05	**0.3895**
Expanding W. 3	0.9925	1.3187	**4.3048**	1.90E−05	0.3663	0.9922	1.5582	**4.5373**	2.36E−05	0.4156
Expanding W. 5	0.9922	1.2462	4.9416	1.88E−05	0.3495	0.9921	1.4754	5.2375	2.35E−05	0.3952
Expanding W. 10	0.9926	1.3221	4.9048	1.89E−05	0.3667	0.9922	1.5617	5.1896	2.36E−05	0.4161
Expanding W. 15	0.9926	1.3186	4.9873	1.89E−05	0.3661	0.9922	1.5583	5.2790	2.35E−05	0.4154
Expanding W. 30	0.9926	1.3168	5.0365	1.88E−05	0.3657	0.9922	1.5557	5.3337	2.33E−05	0.4150
Expanding W. 45	0.9926	1.3191	5.1781	1.88E−05	0.3661	0.9922	1.5587	5.4860	2.34E−05	0.4155

Bold values indicate the minimum values of error metrics

Table 19 Loss functions of rolling window methods on ETH dataset with a 90:10 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	0.9958	1.4570	4.6641	3.98E−06	0.3442	0.9959	1.7416	4.8549	3.61E−06	0.3930
Rolling Window 3	0.9959	1.4270	5.2331	3.92E−06	0.3395	0.9963	1.7072	5.4456	3.55E−06	0.3876
Rolling W. 5	**0.6401**	1.4281	5.0747	3.92E−06	−0.0110	**0.6078**	1.7086	5.2886	3.55E−06	−0.0111
Rolling W. 10	0.6407	1.4291	5.0994	3.92E−06	−0.0110	0.6088	1.7096	5.3109	3.55E−06	−0.0111
Rolling W. 15	0.6539	1.4294	5.3342	3.93E−06	−0.0110	0.6183	1.7100	5.5667	3.55E−06	−0.0111
Rolling W. 30	0.6583	1.4293	5.3205	3.93E−06	−0.0110	0.6218	1.7099	5.5220	3.55E−06	−0.0111
Rolling W. 45	0.7259	**1.0755**	**4.0816**	**3.57E−06**	**−0.0110**	0.6625	**1.2989**	**4.2041**	**3.08E−06**	**−0.0112**
Expanding W. 3	0.9960	1.2973	5.1089	3.68E−06	0.3108	0.9961	1.5557	5.3337	3.27E−06	0.3547
Expanding W. 5	0.9955	1.2702	5.2248	3.71E−06	0.3194	0.9960	1.5245	5.4552	3.31E−06	0.3640
Expanding W. 10	0.9959	1.4491	4.8481	3.96E−06	0.3427	0.9960	1.7326	5.0442	3.59E−06	0.3914
Expanding W. 15	0.9957	1.3068	5.2603	3.72E−06	0.3226	0.9961	1.5672	5.4984	3.30E−06	0.3664
Expanding W. 30	0.9959	1.4525	4.8695	3.97E−06	0.3436	0.9960	1.7366	5.0676	3.60E−06	0.3924
Expanding W. 45	0.9959	1.4556	4.9525	3.97E−06	0.3437	0.9960	1.7401	5.1601	3.60E−06	0.3926

Bold values indicate the minimum values of error metrics

Table 20 Loss functions of rolling window methods on BNB dataset with a 70:30 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	**0.6091**	2.1878	2.8455	3.07E−05	**0.4998**	**0.6087**	2.5325	2.9539	3.44E−05	**0.5525**
Rolling W. 3	0.9758	**2.0909**	2.7555	2.86E−05	0.5093	0.9763	2.4246	2.7796	3.24E−05	0.5577
Rolling W. 5	0.9877	2.1709	4.4225	3.22E−05	0.5367	0.9873	2.5172	4.5533	3.57E−05	0.5853
Rolling W. 10	0.9757	2.1284	3.4973	3.12E−05	0.5066	0.9762	2.4643	3.5846	3.48E−05	0.5555
Rolling W. 15	0.9857	2.1482	4.1057	2.79E−05	0.5238	0.9854	2.4889	4.2253	3.14E−05	0.5700
Rolling W. 30	0.9855	2.1315	4.9056	**2.41E−05**	0.5159	0.9861	2.4718	5.0780	**2.76E−05**	0.5643
Rolling W. 45	0.9855	2.1464	4.8685	2.71E−05	0.5187	0.9852	2.4881	5.0292	3.06E−05	0.5674
Expanding W. 3	0.9847	2.1608	**0.6173**	3.44E−05	0.5419	0.9845	2.4912	**0.6173**	3.80E−05	0.5920
Expanding W. 5	0.6232	2.0929	2.2498	3.28E−05	0.5431	0.6227	**2.4214**	2.2505	3.63E−05	0.5938
Expanding W. 10	0.9885	2.3648	3.8143	3.91E−05	0.5443	0.9890	2.7171	3.9003	4.27E−05	0.5943
Expanding W. 15	0.9886	2.3608	3.9176	3.51E−05	0.5397	0.9892	2.7139	4.0350	3.87E−05	0.5920
Expanding W. 30	0.9888	2.3509	4.0274	3.22E−05	0.5386	0.9892	2.7048	4.1471	3.58E−05	0.5914
Expanding W. 45	0.9884	2.3650	3.9855	3.48E−05	0.5381	0.9892	2.7184	4.1036	3.83E−05	0.5905

Bold values indicate the minimum values of error metrics

and QLIKE functions exhibited inconsistency across the two RV formulas. Notably, in this scenario, no expanding window methods were chosen for either formula.

In the scenario where the entire dataset of Ethereum was partitioned into 70% for training and 30% for testing, an inclination toward rolling window methods, particularly with smaller window sizes such as three and five, can be seen. For both realized volatility formulas, the optimal method based on HMSE, MASE, and MSE loss functions is rolling window 3. However, concerning MAPE, one-step-ahead proves to be the superior approach. Additionally, the outcomes, akin to those in the Bitcoin dataset, exhibit inconsistency among realized volatility formulas in terms of QLIKE. As depicted in Table 17, expanding window methods, akin to the 80:20 scenario in the Bitcoin dataset, manifest inferior performance compared to rolling window methods.

Although the loss functions other than QLIKE indicate the same method for the two formulas at the 80:20 train-test ratio for the ETH dataset, a different method was chosen for each loss function. For instance, in the case of the rolling window 5 method, the HMSE value is 0.9912, representing the lowest value. On the other hand, for the MAPE, the minimum value belongs to rolling window 30 with 1.1612 in Table 18.

In the final Ethereum scenario, the loss functions calculated using the squared five-minute returns and the median realized volatility pointed to the same rolling window methods. Despite rolling window 5 being selected based on HMSE, other loss functions consistently indicated rolling window 45 as the optimal method. This outcome stands out as the most robust and reliable result across different scenarios. Additionally, this scenario marks the third instance where rolling window 45 has consistently delivered superior results across multiple loss functions. Table 19 highlights that the expanding window method, in contrast, failed to yield the best outcomes in any of the examined loss functions.

The results obtained with a train-test ratio of 70:30 for the third and final dataset, the Binance Coin dataset, are presented in Table 20. According to these results, for the one-step-ahead, the HMSE and QLIKE values were calculated as 0.6091 and 0.4998, respectively, in the first formula, and 0.6087 and 0.5525 in the second formula, establishing it as the superior method. In terms of MAPE, the first formula yielded the lowest value on rolling window 3, while the second formula achieved this on expanding window 5. The MASE and MSE error metrics, on the other hand, dictated the selection of the expanding window 3 and rolling window 30 methods.

In Table 21, in the second scenario of the Binance Coin dataset, the expanding window 3 method was chosen concerning MAPE and MASE loss functions. The calculated value of the HMSE based on the square of five-minute returns for rolling window 45 was 0.9867, representing the minimum value. Meanwhile, the corresponding value for the median of realized volatility was computed as 0.9859. While rolling window 30 gave the best results in terms of MSE and QLIKE in the first formula, QLIKE got the smallest value in rolling window 5 in the second formula.

Table 22, which is the final table displaying the results of rolling window approaches, contains the results of the models applied in Binance Coin with a train-test ratio of 90:10. In this table as well, the rolling window 45 method has been identified as the most effective approach in terms of HMSE, MAPE, MSE,

Table 21 Loss functions of rolling window methods on BNB dataset with a 80:20 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	0.9892	1.9564	3.6873	2.81E−05	0.4854	0.9894	2.2415	3.8439	3.35E−05	0.5377
Rolling W. 3	0.9872	1.9629	4.3766	2.67E−05	0.4852	0.9866	2.2715	4.5153	3.21E−05	0.5336
Rolling W. 5	0.9885	1.9485	4.8991	2.68E−05	0.4848	0.9877	2.2587	5.1193	3.20E−05	**0.5323**
Rolling W. 10	0.9869	1.9562	5.0231	2.69E−05	0.4858	0.9865	2.2672	5.2438	3.22E−05	0.5346
Rolling W. 15	0.9878	1.9489	5.0041	2.61E−05	0.4842	0.9871	2.2550	5.2290	3.14E−05	0.5344
Rolling W. 30	0.9869	1.9949	5.0309	**2.56E−05**	**0.4841**	0.9862	2.2967	5.2533	**3.09E−05**	0.5382
Rolling W. 45	**0.9867**	1.9962	5.1329	2.66E−05	0.4875	**0.9859**	2.3062	5.3595	3.18E−05	0.5414
Expanding W. 3	0.9899	**1.9127**	**3.6531**	3.12E−05	0.4886	0.9896	**2.1971**	**3.8155**	3.67E−05	0.5417
Expanding W. 5	0.9897	2.0017	4.2422	3.11E−05	0.4877	0.9895	2.2898	4.4296	3.66E−05	0.5410
Expanding W. 10	0.9896	2.0187	4.1898	2.83E−05	0.4902	0.9893	2.3119	4.3645	3.37E−05	0.5454
Expanding W. 15	0.9897	2.0156	4.1438	3.04E−05	0.4904	0.9895	2.3170	4.3179	3.59E−05	0.5466
Expanding W. 30	0.9904	1.9991	3.7835	2.76E−05	0.4903	0.9903	2.2942	3.9420	3.30E−05	0.5460
Expanding W. 45	0.9904	1.9977	3.7286	2.75E−05	0.4879	0.9904	2.2918	3.8840	3.29E−05	0.5431

Bold values indicate the minimum values of error metrics

Table 22 Loss functions of rolling window methods on BNB dataset with a 90:10 train-test ratio

	Intraday squared returns					Median realized variance				
	HMSE	MAPE	MASE	MSE	QLIKE	HMSE	MAPE	MASE	MSE	QLIKE
One-Step-A	0.9951	1.5970	3.8855	4.70E−06	0.3956	0.9956	1.8699	4.0991	4.38E−06	0.4381
Rolling W. 3	0.9951	1.5457	3.9218	4.52E−06	0.3878	0.9955	1.8101	4.0601	4.18E−06	0.4290
Rolling W. 5	1.0029	1.5455	3.4489	4.49E−06	−0.0109	0.8824	1.8093	3.5818	4.15E−06	−0.0111
Rolling W. 10	0.9932	1.5382	**3.4086**	4.43E−06	−0.0109	0.8760	1.8020	**3.5390**	4.09E−06	−0.0111
Rolling W. 15	0.9924	1.5347	3.9795	4.44E−06	−0.0109	0.8747	1.7965	4.1957	4.10E−06	−0.0111
Rolling W. 30	0.9937	1.5195	3.9667	4.40E−06	−0.0109	0.8771	1.7808	4.1728	4.06E−06	−0.0111
Rolling W. 45	**0.5663**	**1.1449**	4.4267	**3.72E−06**	**− 0.0110**	**0.5952**	**1.3565**	4.5944	**3.29E−06**	**−0.0112**
Expanding W. 3	0.9951	1.5893	3.9539	4.64E−06	0.3949	0.9956	1.8571	4.1722	4.32E−06	0.4371
Expanding W. 5	0.9951	1.5924	3.9681	4.64E−06	0.3951	0.9956	1.8624	4.1878	4.32E−06	0.4376
Expanding W. 10	0.9951	1.5966	4.0300	4.64E−06	0.3953	0.9956	1.8661	4.2530	4.33E−06	0.4377
Expanding W. 15	0.9952	1.5945	3.9169	4.64E−06	0.3952	0.9956	1.8638	4.1377	4.32E−06	0.4373
Expanding W. 30	0.9951	1.6008	3.9973	4.67E−06	0.3965	0.9956	1.8725	4.2161	4.35E−06	0.4389
Expanding W. 45	0.9953	1.6008	4.0163	4.66E−06	0.3963	0.9957	1.8716	4.2364	4.34E−06	0.4388

Bold values indicate the minimum values of error metrics

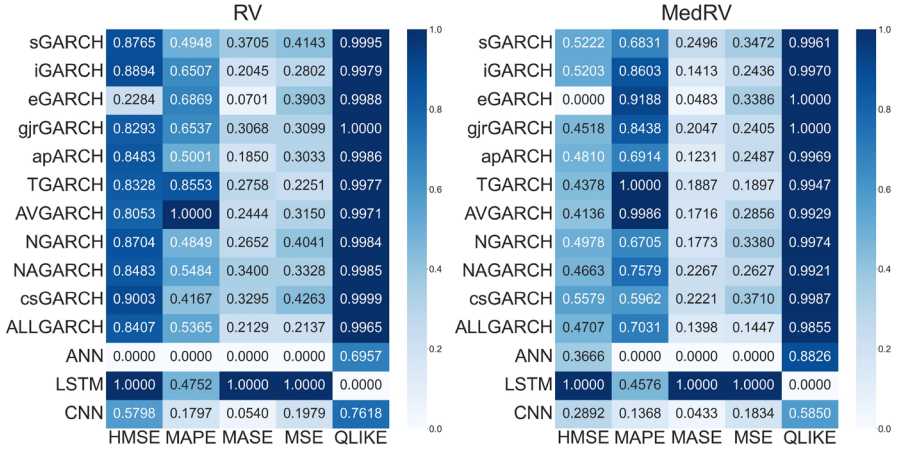


Fig. 11 Heatmap of loss functions on BTC dataset 70:30 train-test ratio

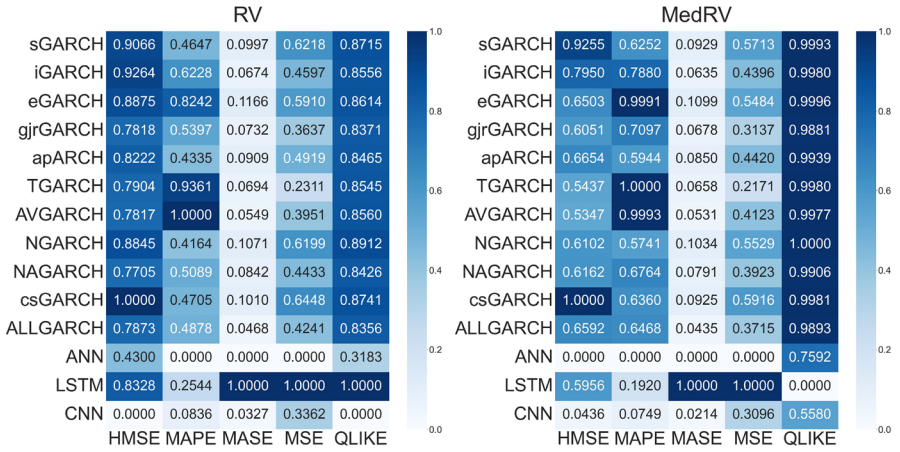


Fig. 12 Heatmap of loss functions on BTC dataset 80:20 train-test ratio

and QLIKE metrics. Meanwhile, rolling window 10 has been determined to be the optimal method concerning the MASE metric.

When a general overview is taken of rolling window methods, it can be stated that as the size of the training set increases, there is a tendency for the loss functions to indicate the same method. Furthermore, it is possible to assert that expanding window methods exhibit poorer performances compared to rolling windows. Models with a window size of 45 in the rolling window methods, especially, demonstrate improved performance as the training set grows. Examination of whether the performances of rolling window methods vary according to realized volatility calculations reveals that, except for QLIKE, there are no frequent inconsistencies across loss functions.

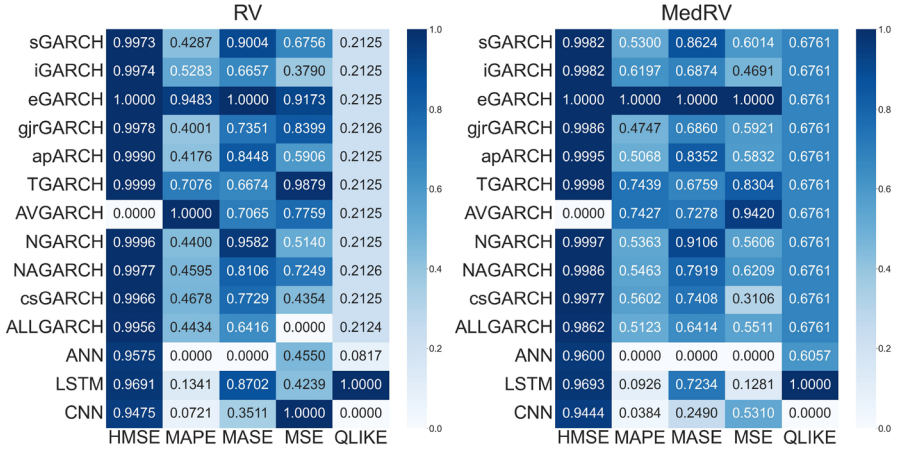


Fig. 13 Heatmap of loss functions on BTC dataset 90:10 train-test ratio

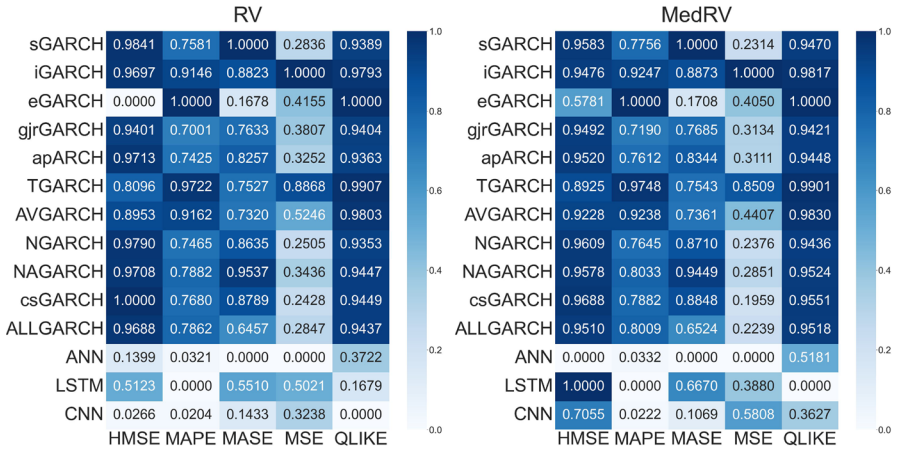


Fig. 14 Heatmap of loss functions on ETH dataset 70:30 train-test ratio

Within the scope of the study, another question under investigation is whether DL methods exhibit superior performance compared to GARCH models. To examine the results from this perspective, heatmaps for various scenarios are provided in Figs. 11, 12, 13, 14, 15, 16, 17, 18, and 19. The heatmaps depict the loss functions of 11 GARCH models and three DL models separately for the squares of five-minute returns and the median realized volatility formulas. For ease of interpretability in the heatmaps, the loss function values were normalized using the minimum-maximum approach, scaling the values to a range between zero and one.

The performance of models with a 70:30 train-test ratio on the Bitcoin dataset is illustrated in Fig. 11. DL models demonstrated superior performance compared to GARCH models in the left graph. Particularly, the ANN model was selected as the

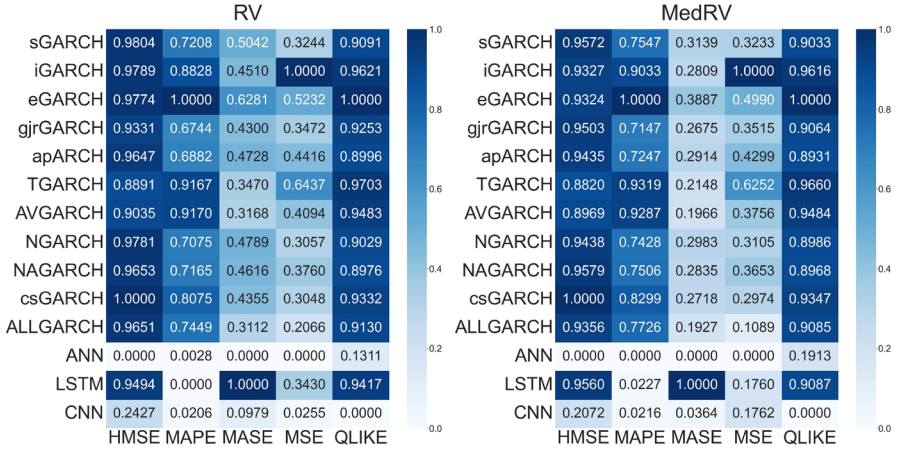


Fig. 15 Heatmap of loss functions on ETH dataset 80:20 train-test ratio

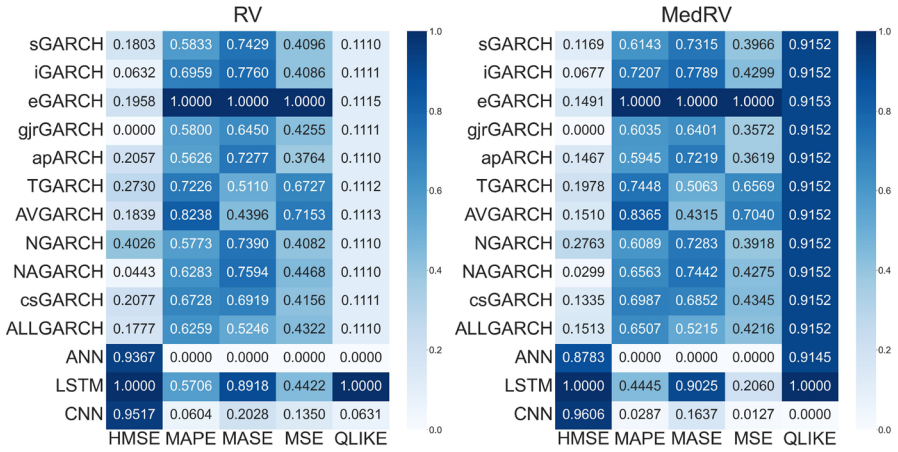


Fig. 16 Heatmap of loss functions on ETH dataset 90:10 train-test ratio

optimal model in terms of loss functions other than QLIKE. However, the selected model for QLIKE was LSTM. In the second realized volatility formula, contrary to the first one, EGARCH was the model that gave the best results based on HMSE.

When examining the loss functions calculated in the Bitcoin dataset with an 80:20 train-test ratio, it can be seen that the ANN model exhibited superior performance compared to other models. The CNN model was selected as the best model based on the first realized volatility formula, based on the HMSE and QLIKE. Another notable observation in Fig. 12 was the discrepancy in the LSTM model's performance, where it was identified as the worst model in terms of QLIKE in the first graph but the best model in the second. This underscores how much the realized volatility formula and loss functions can change the results when evaluating the models.

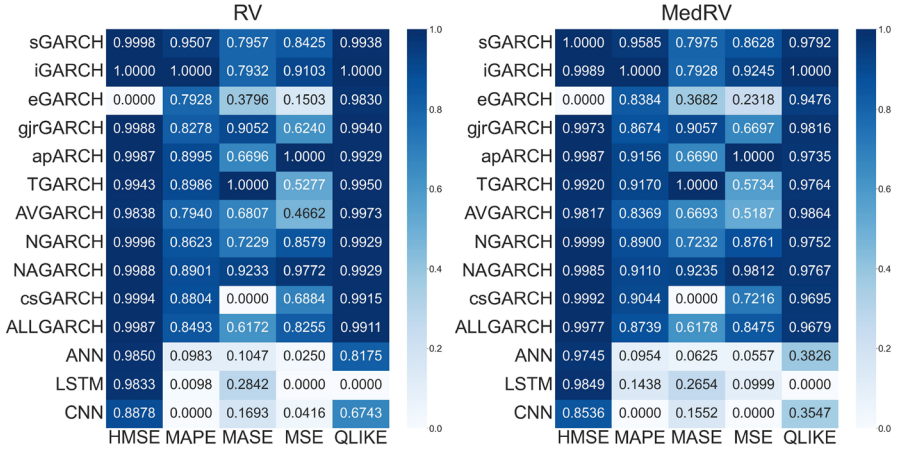


Fig. 17 Heatmap of loss functions on BNB dataset 70:30 train-test ratio

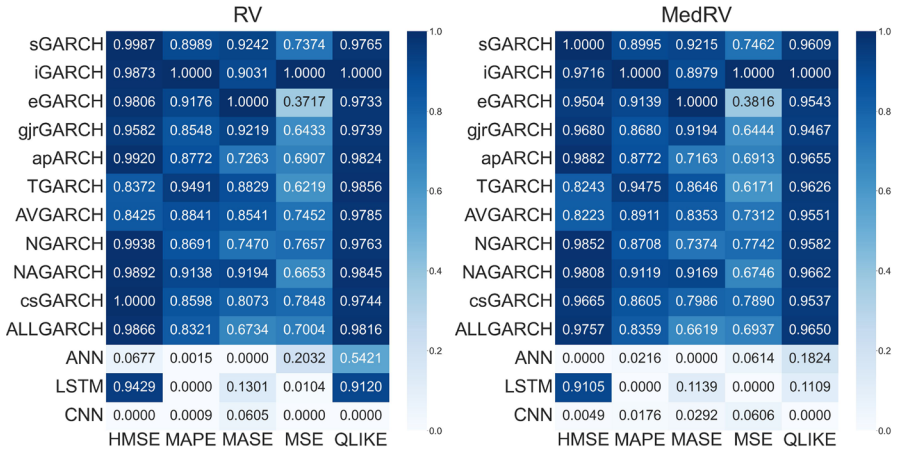


Fig. 18 Heatmap of loss functions on BNB dataset 80:20 train-test ratio

In the last scenario in the Bitcoin data set, a train-test ratio of 90:10 was adopted. According to the findings illustrated in Fig. 13, the AVGARCH model was selected for both realized volatility formulas based on the HMSE criterion. Regarding the QLIKE criterion, the CNN model emerged as the optimal choice, whereas the ALLGARCH model was exclusively chosen based on the MSE in the first graph. In contrast, in the second graph, the ANN model was selected instead of the ALLGARCH model. For the remaining combinations of realized volatility formulas and loss functions, the ANN model was chosen as the optimal one.

In Fig. 14, regarding Ethereum, with a train-test ratio of 70:30, it can be seen that as you go to the lower parts of the graphs where the DL algorithms are located, the colors become lighter, that is, the loss functions take smaller values. EGARCH was

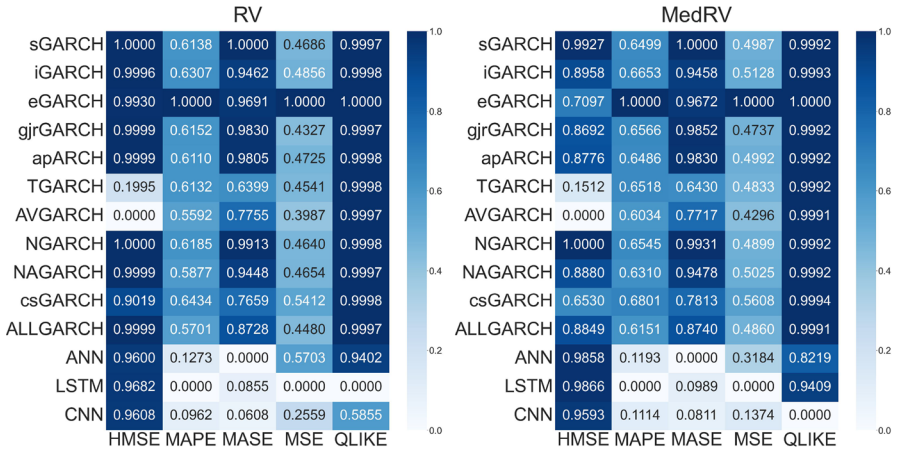


Fig. 19 Heatmap of loss functions on BNB dataset 90:10 train-test ratio

selected in terms of HMSE only in the first formula. For the first formula, two of the remaining loss functions took the smallest value in ANN, one in the LSTM and one in the CNN model. For the second formula, while the first four loss functions pointed to ANN, CNN was the best model only in terms of QLIKE.

The results in Fig. 15 are indicative of the superiority of DL algorithms over GARCH models. In the first graph, the model selected as the best in terms of HMSE, MASE, and MSE was the ANN. In the second graph, it was also chosen as the optimal model based on the MAPE. Regarding the QLIKE, the CNN model was selected in both graphs. LSTM was identified as the DL model selected in the first graph in terms of MAPE.

Figure 16 shows the loss functions of the models in the final scenario for the Ethereum data set. The superiority of the ANN model can be observed in various metrics in both graphs. In the left graph, ANN was selected as the best model based on MAPE, MASE, MSE, and QLIKE. In the right graph, ANN was chosen as the optimal model considering MAPE, MASE, and MSE. GJR-GARCH is only selected in terms of HMSE in the first graph. In the second graph, CNN emerged as the best model regarding QLIKE.

Similar results were obtained in the Binance Coin dataset as in the Bitcoin and Ethereum datasets. Figure 17 depicts the results of the models applied with a 70:30 train-test ratio. In both formulations, the HMSE value points to the EGARCH model, while the MASE value indicates the csGARCH model. For the first formulation, the CNN model was chosen in terms of MAPE, and the LSTM model was selected for MSE and QLIKE. However, in the second formulation, CNN yielded superior results in terms of MSE compared to the LSTM model.

In the examination of the loss functions in the 80:20 scenario of Binance Coin, no GARCH model was selected in the evaluation made using the square of the five-minute returns. Among the DL models, CNN demonstrated superior performance in terms of HMSE, MSE, and QLIKE, as depicted in Fig. 18 on the left. However, due to minor discrepancies among the loss functions, when considering the median

realized volatility, some alterations were observed in the selected models. For instance, on the left, CNN was chosen for HMSE, MSE, and QLIKE, while on the right, ANN was selected for HMSE, and LSTM was chosen for MSE. No changes occurred in the results for other loss functions.

In the last scenario, the models were applied to the Binance Coin dataset in such a way that the training set comprised 90% of the total data. In this scenario, as depicted in Fig. 19, DL models exhibited superior performance regarding loss functions other than HMSE. MAPE, MSE, and QLIKE attained their lowest values in the LSTM model in the first formulation. The ANN emerged as the optimal model for MASE in both formulations, while in the second formulation, CNN took the place of LSTM.

Examining the graphs comparing GARCH models and DL algorithms, it can be seen that the performance of the DL algorithms surpasses that of the GARCH models in numerous scenarios. However, it can be said that the optimal model varies according to the train-test ratio, error function, and realized volatility formula.

4.2 Discussion

According to widely held views, the impact of various parameters, such as forecasting window size, the selection of rolling or expanding windows, the computation of RV measure, and the type of error metric measure, on GARCH-type models under different distributions and deep learning models is a well-known fact. These parameters significantly influence the modeling of volatility and consequently the predictive performance of the models. To strengthen our findings, we conducted an analysis to determine how the best predictive power of 11 GARCH-type models under six different error distributions, and ANN, LSTM, and CNN models perform across different parameters for three cryptocurrency prices, BTC, ETH, and BNB.

The results show that when forecasting out-of-sample returns, considering all five different error measures, the most selected GARCH-type model is the csGARCH model, followed by the ALLGARCH model. Additionally, in terms of error distributions, when examining the five different error measures, the most selected distribution is the ged, followed by the sged distribution.

Considering the two measures used for calculating realized variance and the results involving GARCH-type models, as stated in (Liu et al., 2015), by using 5-minute RV yields, there are more accurate forecast results compared to MedRV across all error measures. However, when considering deep learning models, it has been determined that MedRV is selected more frequently and achieves higher forecast accuracy compared to 5-minute RV.

Another important finding is that the selection of training size improves the forecasts. Considering three different training sizes of 70:30, 80:20, and 90:10, it has been found that both deep learning models and GARCH-type models have higher forecast accuracy when the training/test ratio is 90:10. This result also aligns with the findings of the study conducted by Ewees et al. (2020).

The selection of forecasting window type tested in both GARCH-type models and deep learning models confirms that the rolling forecasting window approach

provides better forecasting performances for three returns. Finally, as stated in the study by Wang et al. (2023), it has been found that deep learning models outperform GARCH-type models in forecasting the volatility of the three returns.

5 Conclusion and Future Work

In recent years, the market capitalization of cryptocurrencies has grown significantly, and they have attracted a lot of interest. Since the first digital money, Bitcoin, was created, there are now many more types. A growing number of businesses and institutions are accepting cryptocurrencies as a result of their rising popularity, which has increased their use in various financial activities. However, compared to other financial assets, cryptocurrencies are more volatile; thus, accurate volatility forecasts are crucial for efficient risk management in the market.

In this study, the volatilities of the three cryptocurrencies with the highest market capitalization were forecasted, and models exhibiting the best out-of-sample forecasting performances were reported. Additionally, the performances of rolling window methods were examined, and whether the results varied across different train-test ratios, realized volatility formulas and loss function combinations was investigated. Eleven distinct GARCH models were considered under six different error distributions. Each GARCH model was implemented using two sliding window techniques with six different lengths. DL algorithms, including ANN, LSTM, and CNN, were modeled with varying delays in series and different numbers of neurons in hidden layers. All the models were applied using three train-test ratios: 70:30, 80:20, and 90:10. Furthermore, two different realized volatility formulas and five different loss functions were employed to examine the results. In total, considering all the datasets and scenarios, a total of 61722 models were implemented within the scope of the study.

When the GARCH models are compared with each other, it can be seen that consistently identifying a superior model or error distribution across all scenarios is challenging. Results vary based on the utilized data set, the train-test ratio, the rolling window technique, the realized volatility formula, and the loss function. Upon examination of results based on rolling window methods, it is generally noted that the performance of the expanding window method is worse than the rolling window method. The most frequently chosen window size is 45. Another finding derived from the performance of the rolling window methods is that the QLIKE loss function is more sensitive to the selected realized volatility formula compared to other loss functions.

When examining the results of DL algorithms, it can be seen that the model with the best performance was ANN, followed by the second-best model, LSTM. CNN, on the other hand, exhibited the poorest performance among the DL algorithms. The outcomes displayed partial differences in terms of the applied volatility formulas. However, when the median realized volatility formula was employed, lower loss functions were obtained compared to the five-minute squared returns formula. Finally, upon comparing the performances of all employed DL algorithms and GARCH models, the DL algorithms exhibited a superior performance. Furthermore,

it is noteworthy that the results vary based on the utilized train-test ratio, realized volatility formula, and loss functions, necessitating distinct examinations for each scenario.

In future work, the forecasting performance of hybrid models, created by combining various commonly used models, can be explored and analyzed. Additionally, the accuracy of forecasts could be improved by developing a combining algorithm with a dynamic structure for combining forecasts from different models.

Acknowledgements We would like to thank reviewers and the editor of the journal for commitment and valuable comments to improve earlier version of this paper.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK). The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Adhikari, R., & Agrawal, R. (2014). A combination of artificial neural network and random walk models for financial time series forecasting. *Neural Computing and Applications, 24*, 1441–1449.

Akyildirim, E., Goncu, A., & Sensoy, A. (2021). Prediction of cryptocurrency returns using machine learning. *Annals of Operations Research, 297*, 3–36.

Amirshahi, B., & Lahmiri, S. (2023). Hybrid deep learning and garch-family models for forecasting volatility of cryptocurrencies. *Machine Learning with Applications, 12*, 100465.

Andersen, T. G., & Bollerslev, T. (1998). Answering the skeptics: Yes, standard volatility models do provide accurate forecasts. *International Economic Review, 39*(4), 885–905.

Andersen, T. G., Dobrev, D., & Schaumburg, E. (2012). Jump-Robust volatility estimation using nearest neighbor truncation. *Journal of Econometrics, 169*(1), 75–93.

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks, 5*(2), 157–166.

Bergsli, L. Ø., Lind, A. F., Molnár, P., & Polasik, M. (2022). Forecasting volatility of bitcoin. *Research in International Business and Finance, 59*, 101540.

Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics, 31*(3), 307–327.

Bollerslev, T. (1987). A conditionally heteroskedastic time series model for speculative prices and rates of return. *The Review of Economics and Statistics, 69*(3), 542–547.

Bouoiyour, J., & Selmi, R. (2015). Bitcoin price: Is it really that new round of volatility can be on way? MBRA Paper No. 65580. University Library of Munich, Germany.

Bouoiyour, J., Selmi, R., et al. (2016). Bitcoin: A beginning of a new phase. *Economics Bulletin, 36*(3), 1430–1440.
Chen, Z., & Jin, S. (2023). Modelling returns volatility: Mixed-frequency model based on momentum of predictability. *Economic research-Ekonomska istra ž ivanja, 36*(1), 2117228.
Di-Giorgi, G., Salas, R., Avaria, R., Ubal, C., Rosas, H., & Torres, R. (2023). Volatility forecasting using deep recurrent neural networks as garch models. *Computational Statistics*, 1–27. https://doi.org/10.1007/s00180-023-01349-1
Dyhrberg, A. H. (2016). Bitcoin, gold and the dollar-a garch volatility analysis. *Finance Research Letters, 16*, 85–92.
ElBahrawy, A., Alessandretti, L., Kandler, A., Pastor-Satorras, R., & Baronchelli, A. (2017). Evolutionary dynamics of the cryptocurrency market. *Royal Society Open Science, 4*(11), 170623.
Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica, 50*(4), 987–1007.
Ewees, A. A., Abd Elaziz, M., Alameer, Z., Ye, H., & Jianhua, Z. (2020). Improving multilayer perceptron neural network using chaotic grasshopper optimization algorithm to forecast iron ore price volatility. *Resources Policy, 65*, 101555.
Fung, K., Jeong, J., & Pereira, J. (2022). More to cryptos than bitcoin: A garch modelling of heterogeneous cryptocurrencies. *Finance Research Letters, 47*, 102544.
García-Medina, A., & Aguayo-Moreno, E. (2024). LSTM-GARCH hybrid model for the prediction of volatility in cryptocurrency portfolios. *Computational Economics, 63*(4), 1511–1542.
Gillitzer, C., & McCarthy, M. (2019). Does global inflation help forecast inflation in industrialized countries? *Journal of Applied Econometrics, 34*(5), 850–857.
Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
Gronwald, M. (2014). The economics of bitcoins–market characteristics and price jumps. CESifo Working Paper Series 5121, CESifo.
He, M., Hao, X., Zhang, Y., & Meng, F. (2021). Forecasting stock return volatility using a Robust regression model. *Journal of Forecasting, 40*(8), 1463–1478.
Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*, 1735–1780.
Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks, 2*, 359–366.
Inoue, A., Jin, L., & Rossi, B. (2017). Rolling window selection for out-of-sample forecasting with time-varying parameters. *Journal of Econometrics, 196*(1), 55–67.
Khaldi, R., El Afia, A., & Chiheb, R. (2019). Forecasting of BTC volatility: Comparative study between parametric and nonparametric models. *Progress in Artificial Intelligence, 8*, 511–523.
Kumar, J., Jilowa, A.K., & Deokar, M. (2024). Volatility modeling of cryptocurrency and identifying common garch model. *Communications in Statistics: Case Studies, Data Analysis and Applications*, pp. 1–18.
Lahmiri, S. (2016). Intraday stock price forecasting based on variational mode decomposition. *Journal of Computational Science, 12*, 23–27.
Lahmiri, S. (2018). Minute-ahead stock price forecasting based on singular spectrum analysis and support vector regression. *Applied Mathematics and Computation, 320*, 444–451.
Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE, 86*(11), 2278–2324.
Lei, L. (2018). Wavelet neural network prediction method of stock price trend based on rough set attribute reduction. *Applied Soft Computing, 62*, 923–932.
Li, Y., Luo, L., Liang, C., & Ma, F. (2023). The role of model bias in predicting volatility: Evidence from the us equity markets. *China Finance Review International, 13*(1), 140–155.
Liu, L. Y., Patton, A. J., & Sheppard, K. (2015). Does anything beat 5-minute rv? a comparison of realized measures across multiple asset classes. *Journal of Econometrics, 187*(1), 293–311.
Maciel, L. (2021). Cryptocurrencies value-at-risk and expected shortfall: Do regime-switching volatility models improve forecasting? *International Journal of Finance & Economics, 26*(3), 4840–4855.
Mallqui, D. C., & Fernandes, R. A. (2019). Predicting the direction, maximum, minimum and closing prices of daily bitcoin exchange rate using machine learning techniques. *Applied Soft Computing, 75*, 596–606.
McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics, 5*, 115–133.
Murray, K., Rossi, A., Carraro, D., & Visentin, A. (2023). On forecasting cryptocurrency prices: A comparison of machine learning, deep learning, and ensembles. *Forecasting, 5*(1), 196–209.

Naimy, V., Haddad, O., Fernández-Avilés, G., & El Khoury, R. (2021). The predictive capacity of GARCH-type models in measuring the volatility of crypto and world currencies. *PloS One, 16*(1), e0245904.

Nelson, D. B. (1991). Conditional heteroskedasticity in asset returns: A new approach. *Econometrica, 59*(2), 347–370.

Panagiotidis, T., Papapanagiotou, G., & Stengos, T. (2022). On the volatility of cryptocurrencies. *Research in International Business and Finance, 62*, 101724.

Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics, 160*(1), 246–256.

Phaladisailoed, T., & Numnonda, T. (2018). Machine learning models comparison for bitcoin price prediction, in *2018 10th International Conference on Information Technology and Electrical Engineering (ICITEE)*, pp. 506–511.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review, 65*(6), 386–408.

Souto, H. G., & Moradi, A. (2024). Introducing NBEATSX to realized volatility forecasting. *Expert Systems with Applications, 242*, 122802.

Wang, Y., Andreeva, G., & Martin-Barragan, B. (2023). Machine learning approaches to forecasting cryptocurrency volatility: Considering internal and external determinants. *International Review of Financial Analysis, 90*, 102914.

Zhang, Y., Ma, F., & Liao, Y. (2020). Forecasting global equity market volatilities. *International Journal of Forecasting, 36*(4), 1454–1475.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.